

Executive Summary

Daily walkers and cyclists increasingly seek routes that prioritise safety, comfort, and overall quality rather than simply minimising time and distance. However, most traditional routefinding applications focus primarily on efficiency metrics, often overlooking critical environmental and safety-related factors such as lighting, crime levels, greenery, and elevation. We have developed a web-based routefinding application designed to enhance traditional navigation by incorporating qualitative route indicators into route computations. For our first prototype, we focused on implementing the basic functionality of the design. The user is able to select a start and end destination on an interactive map of Exeter with visible location nodes that can be clicked on. The application generates and ranks three alternative routes between these points using a numerical route score derived from multiple indicators such as greenery, elevation, light levels etc. Users will also be able to carry out “missions” that challenge them to travel according to various preset routes. These missions will increase user engagement as they have an incentive to keep coming back to the app beyond the need to plan a route. Through these missions, we are also promoting awareness about safer routes. The application is available to use as a standard user or as an admin, who is able to view and change the way in which the system works. For example, an admin could change the weights of certain indicators that affect the route score, or they could add/edit mission data across the system.

While the previously mentioned features make up our first prototype, there is still much to be done for prototype 2. Many quality of life features must be added now the main project functionality works, including analytics views and score breakdowns for routes that are designed in an intuitive way for the user to understand exactly how the route algorithm works. We aim to include the ability for a user to view two alternative routes side by side with descriptive information explaining how well they score on certain indicators. This should allow them to properly understand how routes differ based on the measurements we have selected. A log in system will also be fully developed in our second sprint that will also incorporate role based access control as a security measure to protect user accounts and privacy. We also aim to adapt our application to a mobile-friendly layout for higher quality accessibility. Stakeholder consultations should outline key flaws in our design that we will change in order to make our app more interpretable and usable.

This report details the entire process of our sprint 1 development. Our problem identification section defines our problem in greater detail, creating operationalised requirements with priorities. We define a success criteria for our project before decomposing it into subproblems in the design phase. We detail the architecture of our solutions for these subproblems using data dictionaries, dataflow diagrams and written explanations of theory behind the code as well as the practical tools we used for the implementation. In the testing section, we include our test plans along with their results and an evaluation that discusses our main risks and limitations halfway

through the lifespan of our project. Finally, our report ends with a measurable acceptance criteria for sprint 2.

Risks surrounding the project thus far seem to be oriented around possible scope creep. We have had some difficulty in sprint 1 with fulfilling all the requirements we wished to complete by now, and a couple of them have spilled over into sprint 2 as a consequence. Further stakeholder feedback could also leave us with an amount of features to implement that simply isn't within our scope. To combat this, we aim to continue with our weekly scrum meetings where we review the state of the project and adjust our objectives appropriately.

Problem and Requirements

The problem that our project aims to solve is a prominent issue amongst most routefinding applications; more often than not, apps that aim to display the optimal route between two locations fail to take the finer details into consideration beyond simply using time and distance as a factor. People who walk and cycle on a daily basis want to discover higher quality routes that are safer and more enjoyable to complete, and they are willing to travel longer and further in order to satisfy this. Therefore, we aim to build a web application that expands upon the traditional routefinding app by taking several factors into account when calculating the best route. These factors must describe the quality of the route in ways that time and distance fail to do, such as light levels, crime levels, amount of greenery and elevation. This way, we are offering walkers and cyclists the ability to find routes that match all of their needs based on a variety of elements.

This app should greatly benefit the community of walkers/cyclists who depend on pre-existing routefinding apps to efficiently plan their journeys. We are taking the concept already laid down by apps such as Google Maps and enhancing it to suit the needs of walkers who wish to have safer, more comfortable journeys. Hence, the key benefit of this app is found in increasing the comfort and safety of a walker's day-to-day life, while teaching them important lessons about road safety. A potential risk surrounding our solution that must be mitigated is how walkers could come to trust our app to an extent that may be dangerous. While our application will aim to make journeys more comfortable and convenient, it will not be using life info and all users should still proceed with caution. We will mitigate this issue by ensuring the user is aware of the fact that they shouldn't rely on the app. This can be done via disclaimers that inform the user the app is not to be used with 100% reliance.

In order to achieve our objective, we have assembled a list of requirements that we aim to meet in order to metricise our success. These requirements were defined from a series of user stories that stem from the main problem. We assigned a priority level to each requirement in order to manage our scope for this first sprint. The priority was decided based on how difficult and time consuming the feature would be

to implement along with how essential it is to the core functionality of the application. Every listed requirement must be met by the end of the project, but for this first sprint we focused on the high priority requirements, as well as some medium priority ones, to ensure that we had an application that functioned at a basic level and delivered on all of the key parts of the concept. By the end of this prototype, the user should be able to use the web application to retrieve at least 3 alternative routes between selected locations on an interactive map. These routes should be ranked via a route score that is calculated from 5 indicator values from a seeded dataset. The table below visualises our requirements as well as the user stories they were developed from, their priority level and which sprint they are to be completed in.

Requirement Number	User Story	Requirement Description	Priority	Sprint to Be Completed In
1	As a user, I would like to be able to visualise routes on the map so that I can understand them better.	Users can select two locations on an interactive map.	Medium	Sprint 1
2	As a traveller, I want to plan a route between two locations so that I can travel safely.	Compute at least 3 route alternatives between two points.	High	Sprint 1
3	As a traveller, I want a high quality route that takes multiple features into account beyond just distance.	Compute a route scoring using at least 5 indicator values.	High	Sprint 1
4	As a traveller, I want a high quality route that takes multiple features into account beyond just distance.	Show a breakdown of each route indicating how each factor contributed to the score.	Low	Sprint 1
5	As a traveller, I want to be able to plan routes based on my personal preferences towards which	Allow users to adjust weights for at least 4 factors.	Low	Sprint 1

	features matter the most.			
6	As a traveller, I want a high quality route that takes multiple features into account beyond just distance.	Side-by-side comparison of two routes on indicator breakdown and score.	Low	Sprint 2
7	As a traveller, I want a high quality route that takes multiple features into account beyond just distance.	At least 6 analytics views across the system .	Low	Sprint 2
8	As a user, I would like to be able to visualise routes on the map so that I can understand them better.	Route segments should be displayed on the map.	Medium	Sprint 1
9	As a user, I would like to be able to understand how routes are being scored based on their indicators	Route segments should show indicator tooltips upon hovering.	Low	Sprint 2
10	As an admin, I would like to be able to create and change missions so that I can maintain the app.	Admins must be able to create/edit missions.	High	Sprint 1
11	As an admin, I would like to be able to update the weights of indicators to improve the way in which routes are scored.	Admins must be able to update scoring weight defaults and indicator thresholds.	Medium	Sprint 1
12	As an admin, I	Change log	Medium	Sprint 2

	would like to see what changes have been made to the application so I can document which developer is doing what.	required for edits (i.e., who changed what and when).		
13	As an admin, I would like to have a secure application that ensures only authorised individuals can make changes and view private information.	Role based access control.	Medium	Sprint 2
14	As a traveller using a mobile phone, I want to be able to plan routes for myself on my phone with ease.	Mobile-friendly layout.	Medium	Sprint 1
15	As a traveller I want to select routes to complete a mission to learn more about safe routes and to have an incentive to keep using the app.	Users can complete preplanned "missions" between points assigned to them.	High	Sprint 1

In sprint 2, the main focus lies on meeting requirements that need the basic functionality of the app (delivered in prototype 1) to already be completed. We must create an operationalised acceptance criteria in order to make the completion of our requirements measurable. The acceptance criteria for the sprint 1 requirements can be found below.

Sprint 1 Acceptance Criteria

Requirement Number	Requirement Description	Measurement Method
1	Users can select two locations on an interactive map.	Functional Test: start and end points can be selected without errors.

2	Compute at least 3 route alternatives between two points.	Automated/Manual test verifying that ≥ 3 unique routes are returned.
3	Compute a route scoring using at least 5 indicator values.	Automated/Manual test verifying that a route score is returned when two locations are selected. Code inspection should verify that at least 5 indicator values are used in the scoring function.
8	Route segments should be displayed on the map.	Visual inspection (screenshot) of interactive map rendering routes.
10	Admins must be able to create/edit missions.	Functional test confirming that missions can be created/edited successfully.
14	Mobile-friendly layout.	Visual inspection of the application on a mobile interface, ensuring it is still functional.
15	Users can complete preplanned "missions" between points assigned to them.	Test confirming that users can complete missions.

Furthermore, we must create a success criteria for the entire project by identifying the key measures of success we will use to determine to what extent we have successfully completed our project in the evaluation section. A table describing our measures of success can be found below.

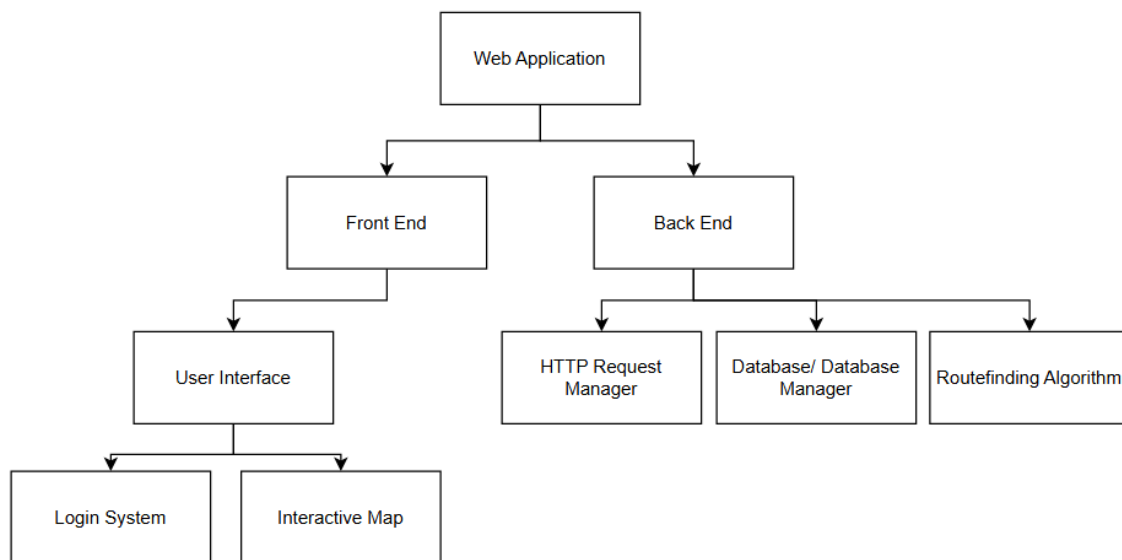
Measure of Success	Description	Type of Evidence
Interpretability	Users must be able to understand how the app works and what factors contribute to route computation.	Stakeholder interview: can our stakeholders explain why a certain route ranked the highest?
Algorithmic Correctness	The algorithm used to calculate the route scores must be consistent and correct.	Audit of route score calculations and weights should demonstrate a consistent algorithm that correctly assigns a score to routes based

		on weight sensitivity.
Engagement	Users must find the app engaging; repeatedly using the features such as route planning and mission completion.	Ensure that missions are being completed by users repeatedly over time.
Fairness and Harm Analysis	The application should not mislead users into trusting the route score as an indicator of complete safety.	Ethical discussion surrounding our application and a justification as to how our User Interface resolves these issues.
Program Functionality	The program should function consistently and reliably, fulfilling the requirements.	Unit testing can be used for some aspects of the program. Whereas comprehensive full test runs can be used to ensure the system works as a whole. All requirements in the acceptance criteria should be met.

Design and Architecture

Problem Decomposition

In our first Scrum meeting, we decided to visualise the problem as a diagram in order to decompose the problem into multiple subproblems that we could assign individual developers to. This way, we improve efficiency by pipelining different tasks that can have their own internal development cycles. This diagram was initially drawn on a whiteboard, but has been recreated digitally to demonstrate the initial breakdown of our project.

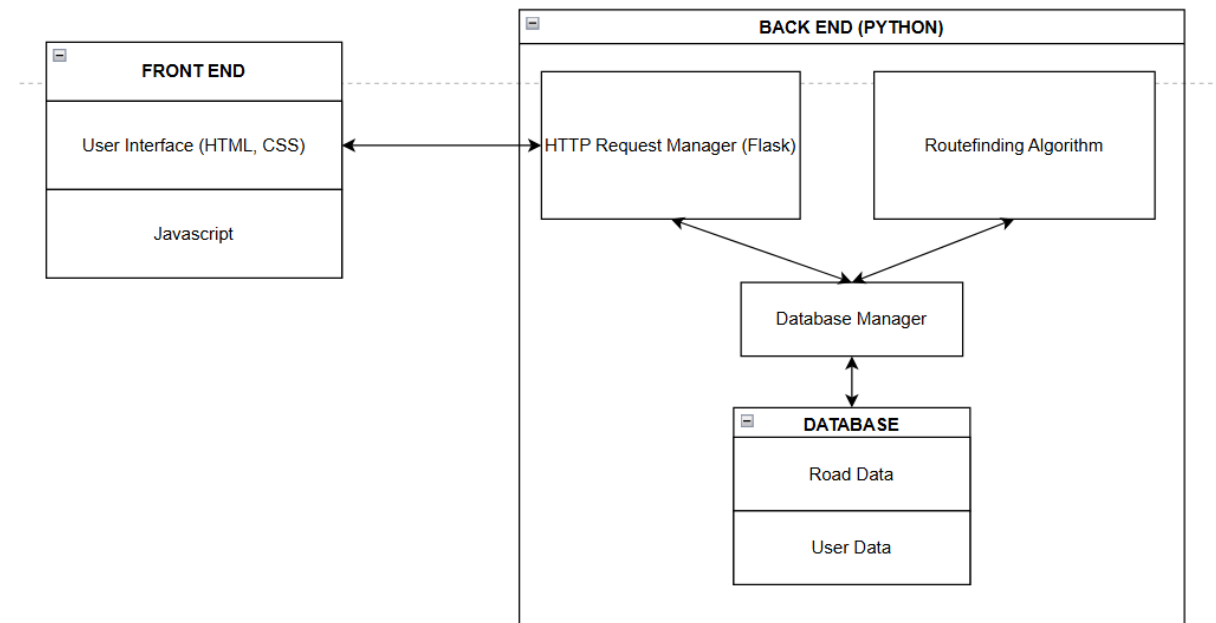


The diagram above breaks our problem into simple components. Some parts, such as the routefinding algorithm, were assigned to a single person whereas more complex problems such as the front end and the HTTP request manager had up to two people working on them. For these components where developers paired up to work, we utilised paired programming in order to maximise the efficiency of our workflow. This involved dividing a pair into a “driver” who actually wrote the code and a “navigator” who would strategise and plan the way in which the problem would be solved logically. These roles would reverse every so often as to keep a fresh perspective and dedicated pace to the development of each component.

We decided that our web application would have to be split into the front end and the back end; the front end would be the user-facing side of the application that featured the user interface. This layer would be responsible for presenting information to the user and capturing their inputs. The interface would consist of a login system where the user could enter their credentials to then be granted access to the interactive map where they could plan our their routes.

The back end features the main functionality of our program, where all of the logic and data processing is carried out. The HTTP Request Manager acts as a bridge

between the front end and the server, taking requests that were results of user input and returning appropriate responses. We also need to have a database to store relevant road and user data that needs to be used in the routefinding algorithm and sent to the frontend. A database manager will be necessary to perform all operations regarding this database, handling queries and making updates. To better understand how these components would actually interact with one another, we created an additional diagram that marked interactions with arrows to visualise where the connections were.

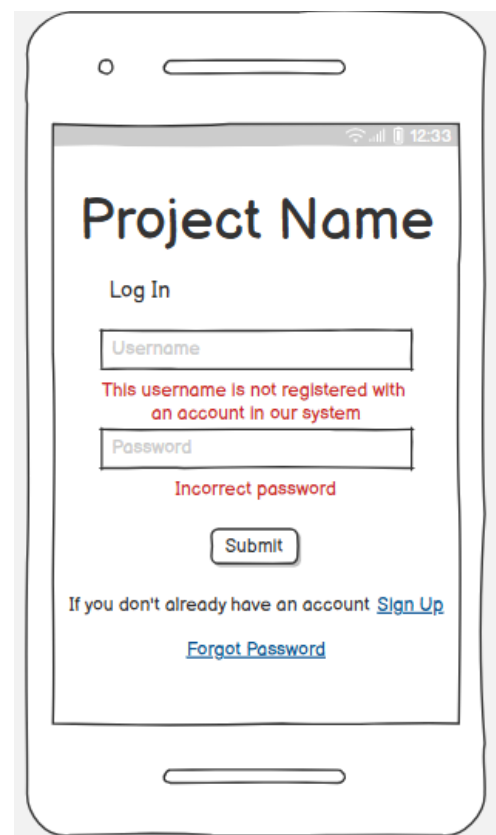
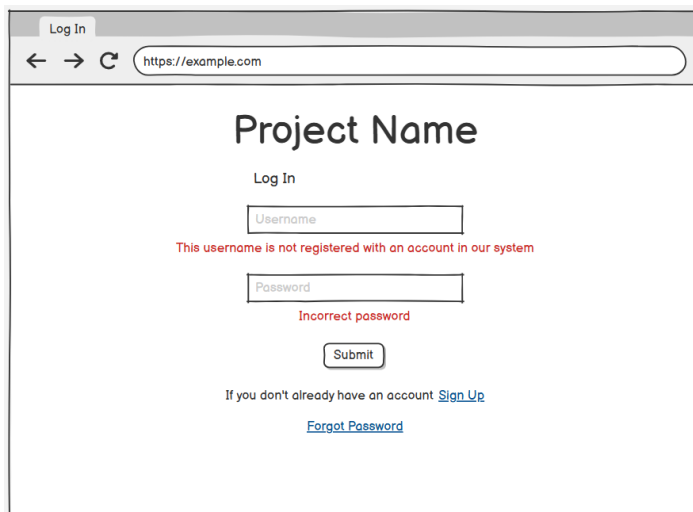
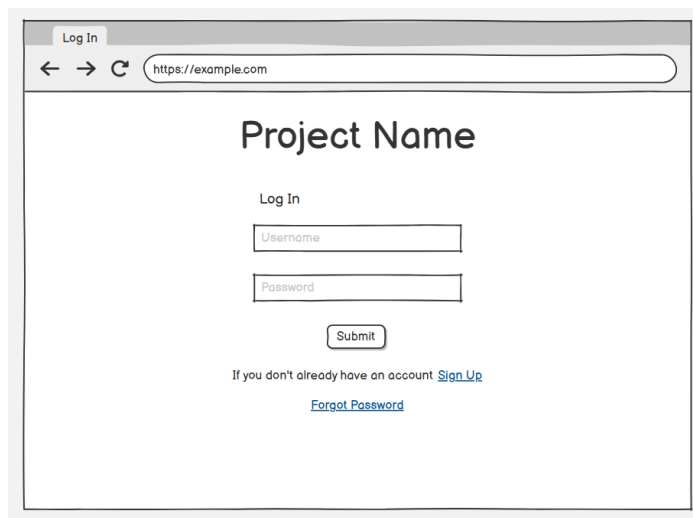


The diagram above helped us understand which developers had to work together when it came to creating their components and ensuring they functioned in unison by the end of the first sprint. The request manager would interact with the front end, also communicating with the database manager to retrieve user and road data. For example, when a user logs into the system their credentials must be validated. This requires the credentials to be sent to the request manager, which will validate the details with the help of the database manager and return a response that either allows or denies the user access to the rest of the application. This concept is explored further on in the report. The routefinding algorithm will also need to interact with the database manager, as it requires data regarding the roads to compute a route score. At this point, we decided that we would use Python for the backend. This is not only because python is familiar amongst all of our developers, but also due to the fact that it is very easy to read and understand, contributing towards our efforts to make our code clear and concise. Python also allows us to use Flask, which is a tool that will be used for the HTTP Request Manager. Using flask will be very efficient as it is simple to use and will reduce the complexity of request handling significantly. The front end will be built using HTML, CSS and JavaScript like most front ends are.

Front End Design

In order to plan the structure of the application, we created several wireframes to visualise the user interface that the user would view and interact with. This greatly helped us when it came to building the front end with html, css and javascript as there was a structure to adhere to. These wireframes can be seen and explained below, and they were made using the tool Balsamiq which increased the efficiency of our workflow due to how it is specialised to help create what we wanted to achieve.

Log In Page:



This is the wireframe for the log in page, it will be the first page that the user is met with when they use the app and will be used to verify the user identity. The design is simple and not overwhelming so that when users are met with this page, they immediately know that they need to input their username and password. If the user does not enter anything or enters the wrong credentials upon a submit, then red

error messages will appear to tell them what they have done wrong, as demonstrated above.

Connections:

Submit button → Main Page

Sign Up link → Sign Up page

Forgot Password link → Forgot Password Page

Sign Up Page

A wireframe of a desktop web browser showing a 'Sign Up' page. The browser's address bar displays 'https://example.com'. The page has a title 'Sign Up' and four input fields labeled 'Email', 'Username', 'Password', and 'Confirm Password' stacked vertically. Below these fields is a 'Submit' button. At the bottom of the form, there is a link that says 'If you already have an account [Log In](#)'.

A wireframe of a mobile smartphone displaying the 'Sign Up' page. The screen shows the title 'Sign Up' and four input fields labeled 'Email', 'Username', 'Password', and 'Confirm Password' stacked vertically. Below these fields is a 'Submit' button. At the bottom of the form, there is a link that says 'If you already have an account [Log In](#)'. The phone's status bar at the top shows signal strength, battery level, and the time '12:49'.

This is the sign up page, which will be used by users to create a new account if they don't already have one. The sign up page will be styled similarly to the log in page because the two are closely related and perform similar actions.

Connections:

Submit button → Log In Page

Log In link → Log In Page

Forgot Password Pages

Page 1

A desktop browser window showing the 'Forgot Password' page. The browser's address bar displays 'https://example.com'. The page has a title 'Log In' in the top left corner. The main heading is 'Forgot Password'. Below it is a text input field labeled 'Email'. A 'Submit' button is positioned below the input field. At the bottom, there is a link that says 'Or [Log In](#)'.

A mobile phone screen displaying the 'Forgot Password' page. The status bar at the top shows the time as 12:34. The page layout is vertically centered. The heading 'Forgot Password' is at the top. Below it is an 'Email' input field. A red error message reads: 'We don't seem to have that email on record. Please try again'. Below the error message is a link: 'or to make an account [Sign Up](#)'. A 'Submit' button is located below the link. At the bottom, there is a link: 'Or [Log In](#)'.

A desktop browser window showing the 'Forgot Password' page with an error message. The browser's address bar displays 'https://example.com'. The page has a title 'Log In' in the top left corner. The main heading is 'Forgot Password'. Below it is a text input field labeled 'Email'. A red error message reads: 'We don't seem to have that email on record, please try again'. Below the error message is a link: 'or to make an account [Sign Up](#)'. A 'Submit' button is positioned below the link. At the bottom, there is a link that says 'Or [Log In](#)'.

Page 2

Log In

https://example.com

Forgot Password

We've sent an email with a code to {email} please enter it below

Submit

Or [Log In](#)

Log In

https://example.com

Forgot Password

We've sent an email with a code to {email} please enter it below

Incorrect Code

Submit

Or [Log In](#)

12:49

Forgot Password

We've sent an email with a code to {email} please enter it below

Submit

Or [Log In](#)

The forgotten password pages are a “could have” feature of our app, it would make sense to have but is not a required feature. They would be used by a user to gain access to their account if they had forgotten their password. These will also be styled similarly to the signup and log in pages.

Connections:

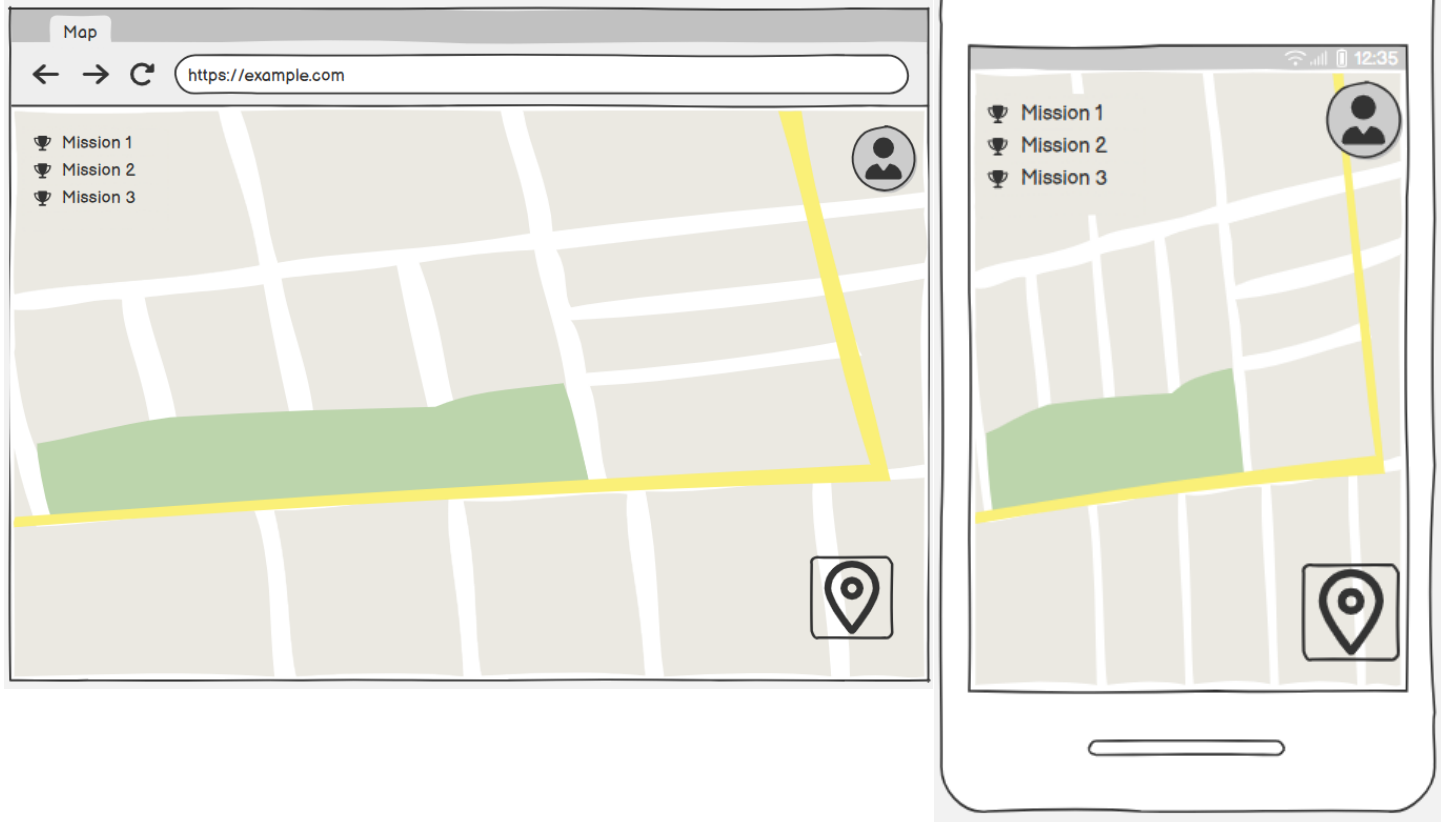
Submit Button 1 → Forgot Password Page 2

Submit Button 2 → Log In Page

Log In link → Log In Page

Sign Up button → Sign Up Page

Main Page



This is the main page of the app where users will spend most of their time. It will display the route they have chosen and provides the buttons to access the rest of the functionality of the app. The map is the main point of this app so it should take up as much of the screen as possible, therefore the buttons are overlaid on top of the map so that no space is wasted. The map background will be interactable, allowing users to tap on locations, scroll to a different area, or zoom in and out. The missions list in the top left will display the missions that the user has currently pinned so that they can be seen more easily from the main page.

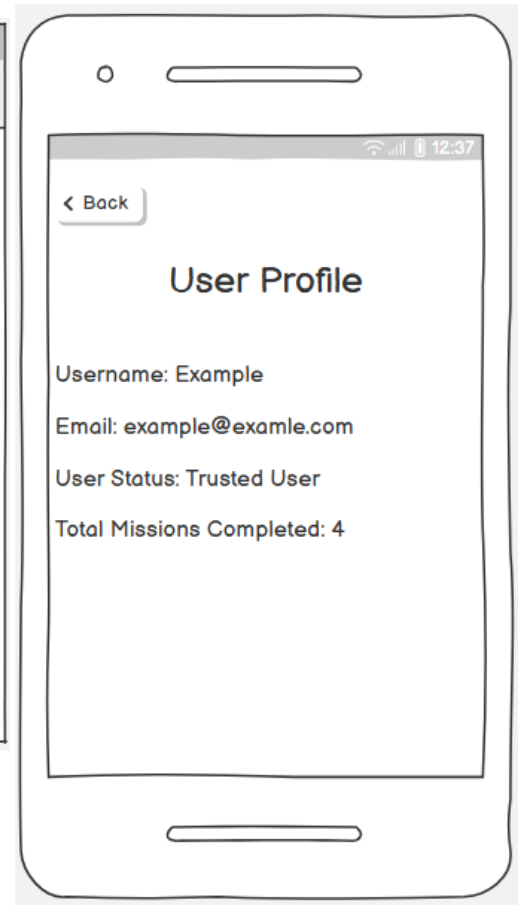
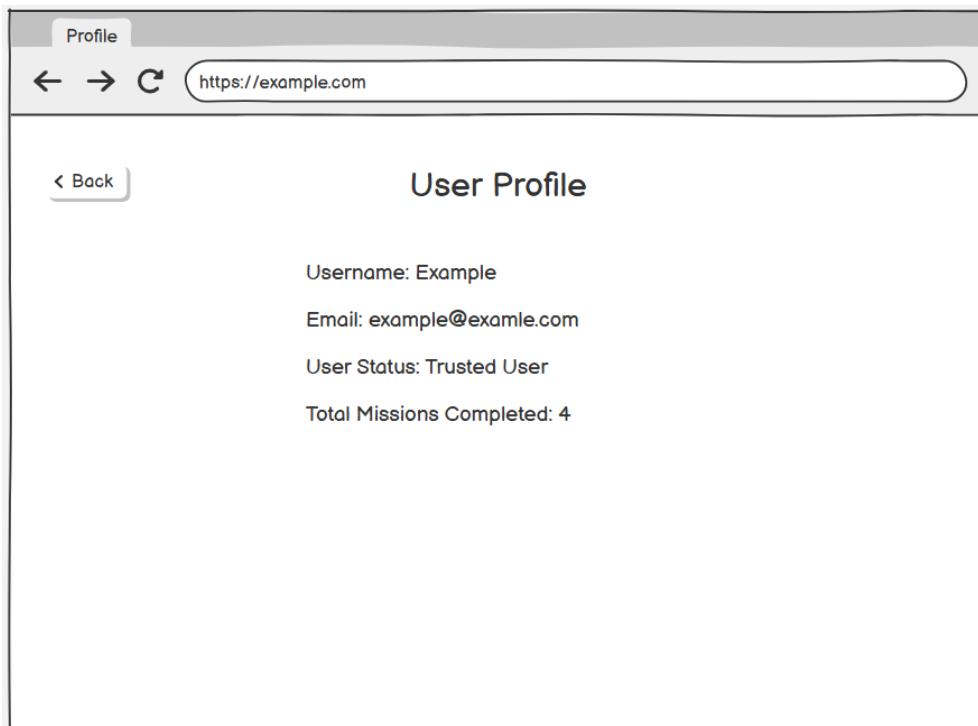
Connections:

Mission list (Top Left) → Missions Page

Profile button (Top Right) → Profile Page

Route button (Bottom Right) → Plan Route Page

Profile Page



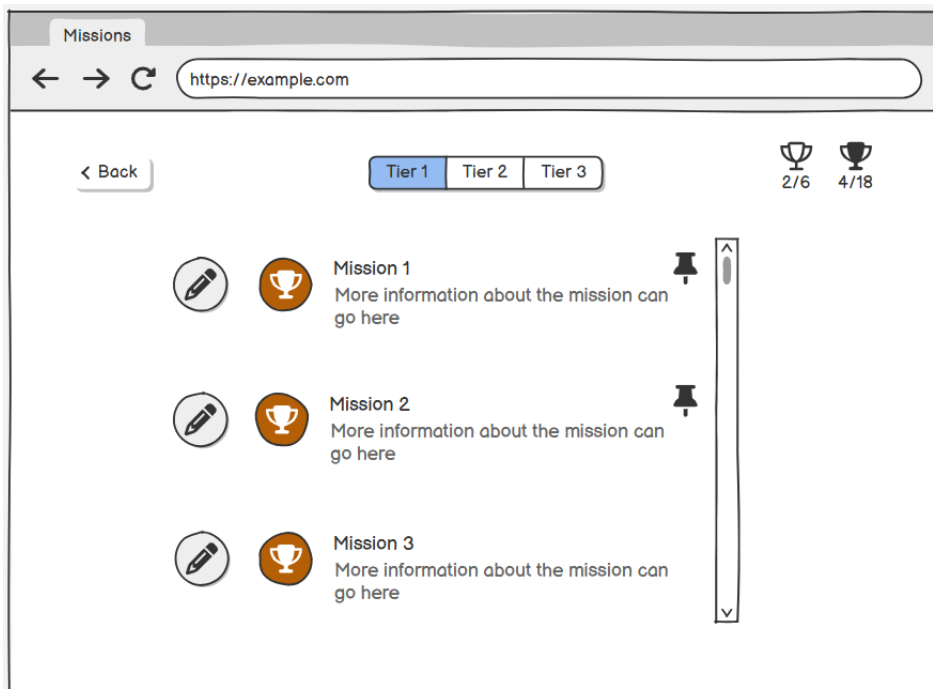
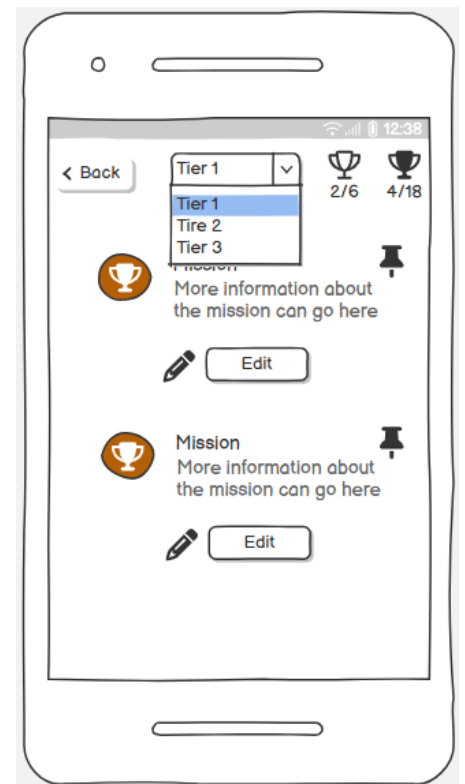
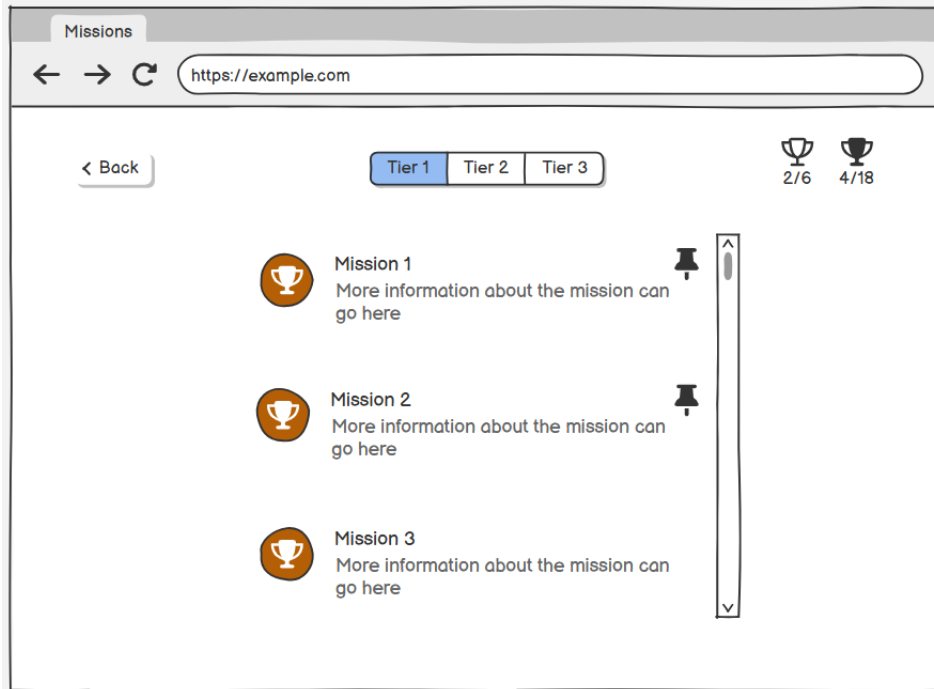
This is the user profile page where users can view information about their account. In the mobile version of this page the text is aligned to the left rather than centred, I believe this is a better look for text on a mobile device.

Connections:

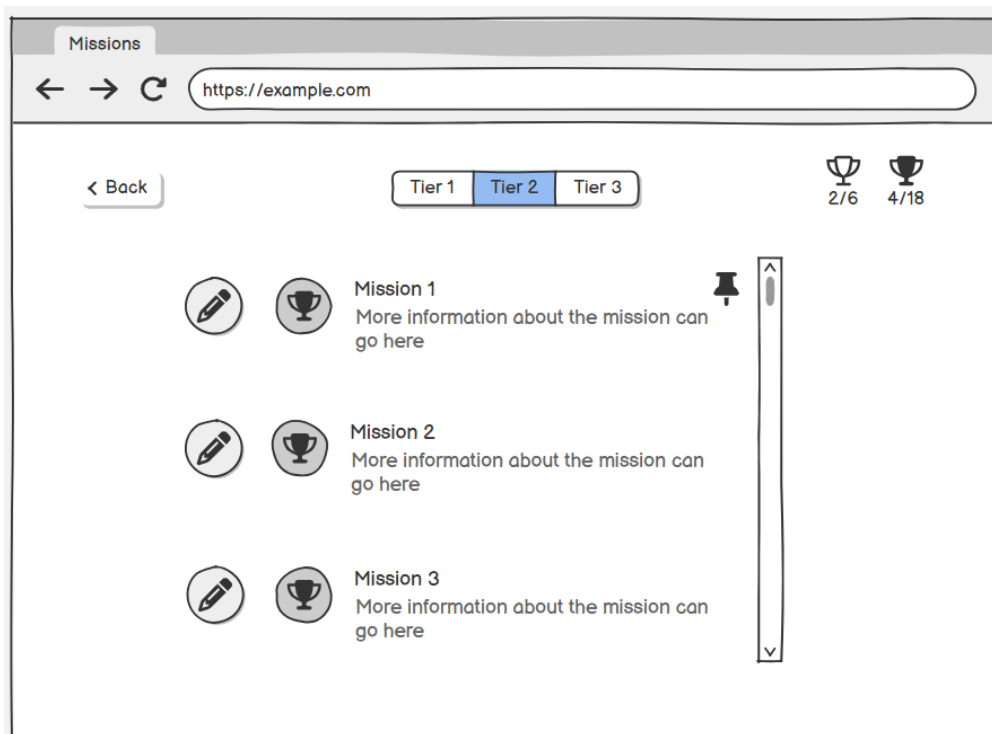
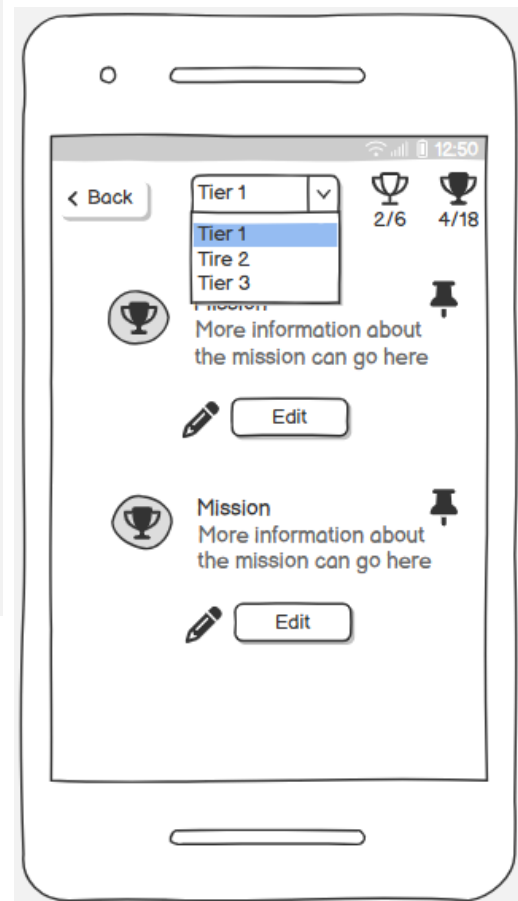
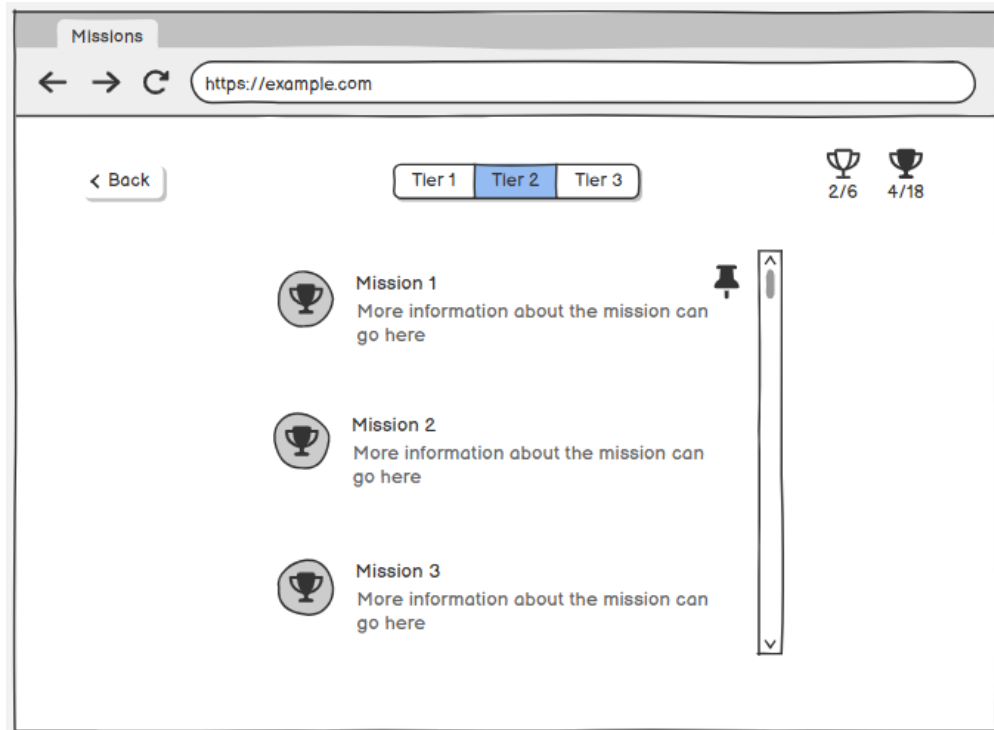
Back button → Main Page

Missions Pages

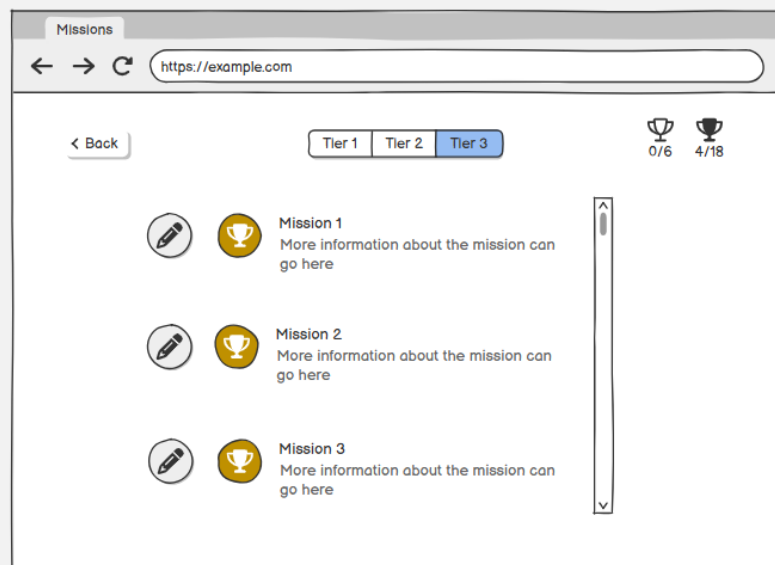
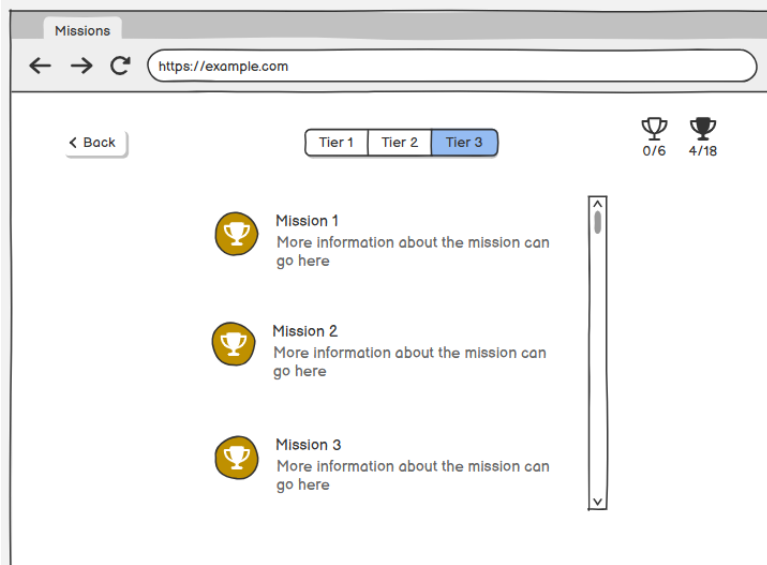
Tier 1



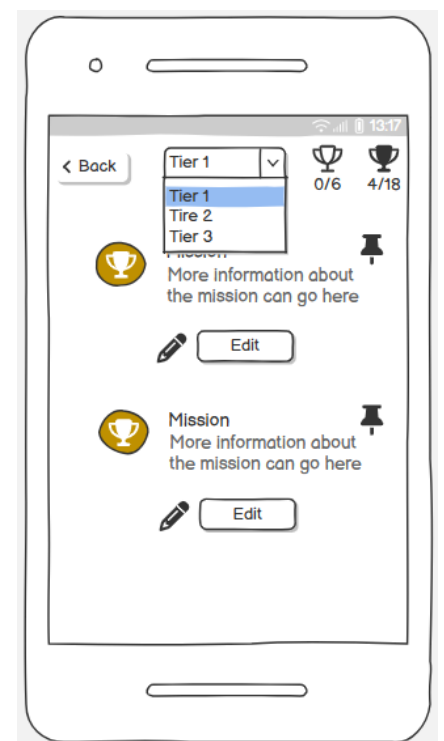
Tier 2



Tier 3



These are the 3 tiers of the mission pages, each display the list of missions that come under that tier. The trophies in the top right show how many missions the user has completed, the white one shows how many have been completed in that tier, and the black one shows the total number that have been completed. An alternate view is available for trusted users which includes the edit button (to the left of each mission, or below on mobile), this will allow trusted users to edit the missions. When a mission is clicked then it will get pinned to the main page, which will make it appear in the missions list in the top left of the main page. The mobile versions of this page have been altered slightly to better fit the proportions of a narrower screen. The top bar has been swapped with a drop down list because the top bar would most likely look squashed on a mobile screen. The edit button has also been moved below the missions for the same reason.



Connections:

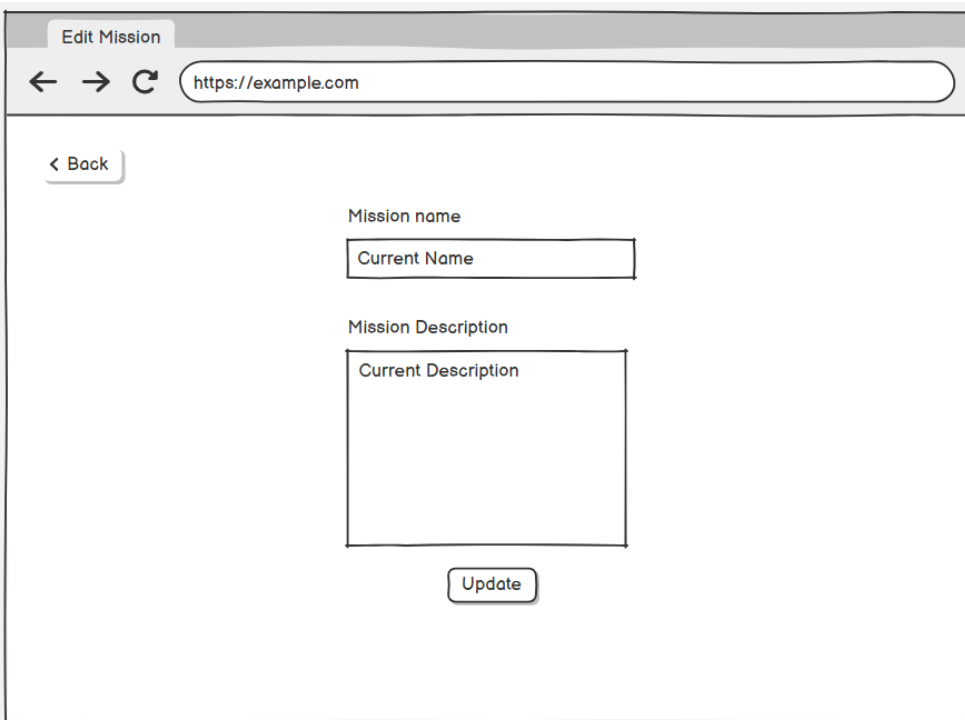
Tier Drop Down/Nav Bar → Used to switch between mission tiers, e.g. clicking on "Tier 1" will take you to the tier 1 page

Back button → Main Page

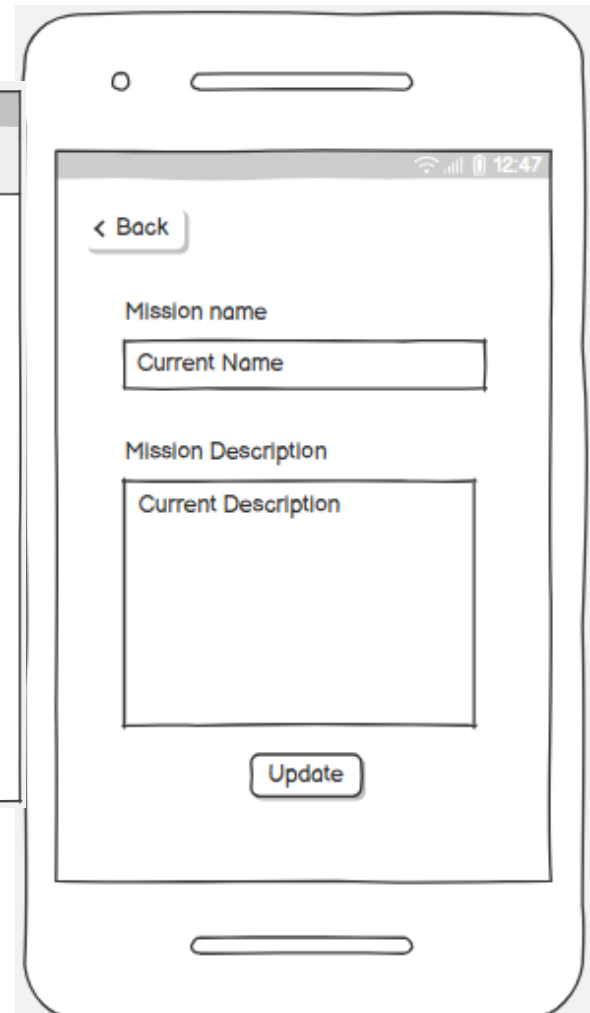
Missions (on click) → Mission will become pinned to the main page

Edit buttons → Edit Mission Page

Edit Mission Page



Desktop view of the Edit Mission page. The browser address bar shows 'https://example.com'. The page title is 'Edit Mission'. The form includes a '< Back' button, a 'Mission name' label above a text input field containing 'Current Name', a 'Mission Description' label above a larger text area containing 'Current Description', and an 'Update' button at the bottom.



Mobile view of the Edit Mission page. The status bar at the top shows signal strength, Wi-Fi, and the time 12:47. The page features a '< Back' button, a 'Mission name' label above a text input field with 'Current Name', a 'Mission Description' label above a text area with 'Current Description', and an 'Update' button at the bottom.

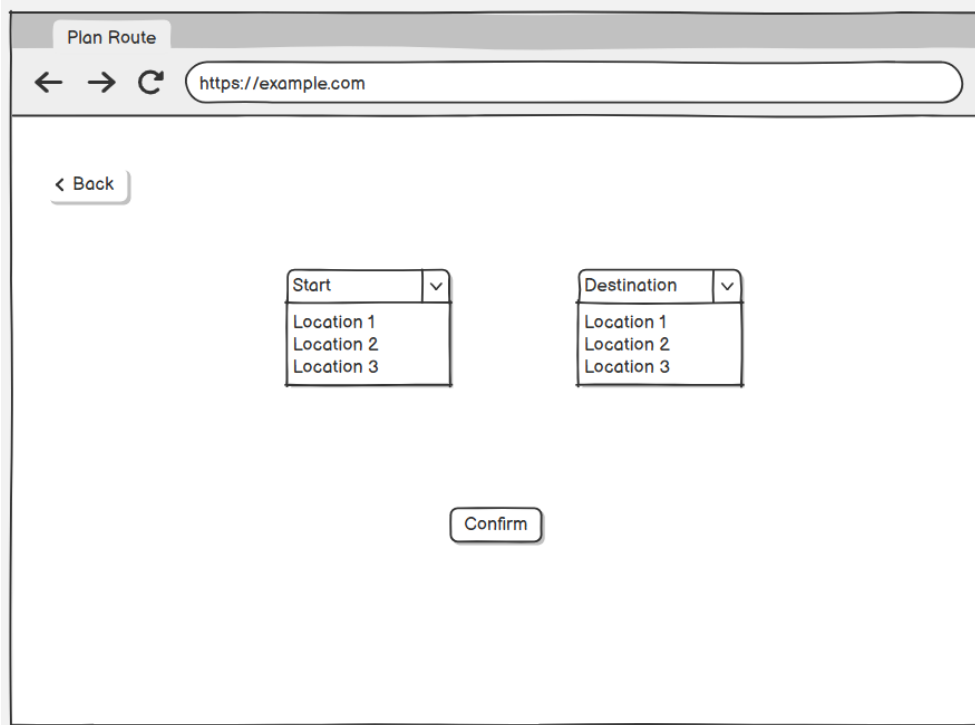
This is the edit mission page, which will be accessed by trusted users only. This allows trusted users to edit the contents of a mission and update them. The text in the input boxes should display the current information about a mission, then the user can change the mission details and hit the update button to save any changes made.

Connections:

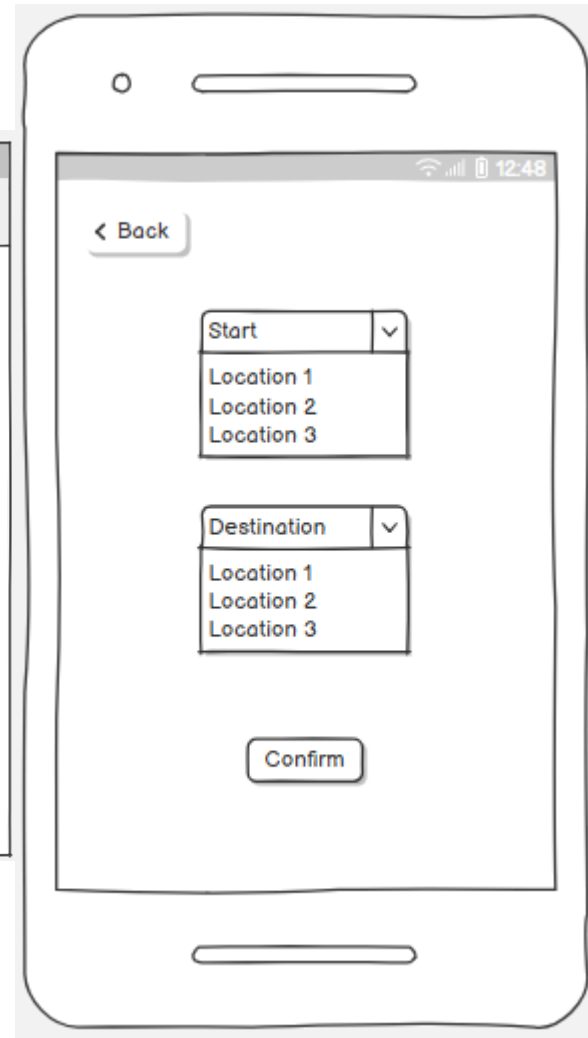
Update button → Return back to previous page

Back button → Return back to previous page

Plan Route Page



The desktop version of the Plan Route page features a browser window with a title bar 'Plan Route' and a URL bar showing 'https://example.com'. The page content includes a '< Back' button in the top left. In the center, there are two side-by-side dropdown menus. The first is labeled 'Start' and the second is labeled 'Destination'. Both menus have a downward arrow icon and a list of three options: 'Location 1', 'Location 2', and 'Location 3'. Below these menus is a 'Confirm' button.



The mobile version of the Plan Route page is displayed on a smartphone screen. It features a status bar at the top with signal strength, battery level, and the time '12:48'. The page content includes a '< Back' button in the top left. Below it, the two dropdown menus are arranged vertically. The first is labeled 'Start' and the second is labeled 'Destination'. Both menus have a downward arrow icon and a list of three options: 'Location 1', 'Location 2', and 'Location 3'. Below these menus is a 'Confirm' button.

This is the plan route page, which allows users to select a start and end destination for their route that will be displayed on the main page. On the mobile version of this page the drop down lists are arranged vertically rather than horizontally, because there is not enough space to arrange them horizontally on a mobile screen.

Connections:

Confirm button → Main Page

Back Button → Main Page

JavaScript Data Dictionary

Variable Name	Description	Data Type	File
username_input	Refers to the username input box on either the log in page or the sign up page	Object	frontend_validation.js
email_input	Refers to the email input box on the sign up page	Object	frontend_validation.js
password_input	Refers to the password input box on either the log in or sign up page	Object	frontend_validation.js
confirm_password_input	Refers to the confirm password input box on the sign up page	Object	frontend_validation.js
error_message	Refers to the paragraph tag on either the log in or sign up page that contains the error message	Object	frontend_validation.js
form	Refers to the entire log in/sign up form	Object	frontend_validation.js
inputs	An array of all the input fields that are relevant depending on whether the user is on the log in page or the sign up page	Object Array	frontend_validation.js
errors	An array of all the error messages that need to be output to the user	String Array	frontend_validation.js
currentPage	Holds the title of the current page	String	missions.js
missions	Is a collection of all the mission elements on the current page	Object Collection	missions.js
pins	Is a collection of all the pin elements on the current page	Object Collection	missions.js
trophys	Is a collection of all the trophy elements on the current page	Object Collection	missions.js
editButtons	Is a collection of all the edit buttons on the current page	Object Collection	missions.js
missionText	Is a collection of all the mission text elements on the current page	Object Collection	missions.js
userClass	Holds the users rank on the website,	String	missions.js

	e.g. "trusted" or "user"		
tier	Stores which mission tier the page is currently on	String	missions.js
pinTier	Stores the key value used to store the pin state array in local storage	String	missions.js
selected	Refers to the currently selected tier button on the navigation bar at the top of the screen	Object	missions.js
pinValues	An array that hold the visibility states of all the pins on the current page	String Array	missions.js
form	Refers to the entire form used to edit the mission	Object	edit_mission.js
update	Refers to the update button on the edit mission page	Object	edit_mission.js
description	Refers to the mission description input box on the edit mission page	Object	edit_mission.js
redButton	Refers to the red button on the mission display screen	Object	mission_display.js
blueButton	Refers to the blue button on the mission display screen	Object	mission_display.js
greenButton	Refers to the green button on the mission display screen	Object	mission_display.js
error	Refers to the error message displayed on the mission display screen	Object	mission_display.js

Back End Design

We started off by using the [Node.js](#) runtime. [Node.js](#) is a free, open-source JavaScript runtime used for creating web applications. We chose [Node.js](#) because of its widespread adoption by major platforms such as Netflix and LinkedIn, and its growing use within the industry. However, during development it became apparent that [Node.js](#) was too large for the scope of the project. The amount of time to learn and set up [Node.js](#) outweighed its benefits.

Since development was still in its early stages and we had found a more lightweight framework, we decided to switch. The framework we moved to is called Flask and uses Python. Flask is a more appropriate choice for this project. Not only is it simpler to use and understand, but also the team is more experienced with Python and Flask with previous modules such as ECM1400 Programming using it. Additionally, Flask

using Python also means it is more compatible with other scripts written in the project such as ones for accessing the database and the route finding algorithm.

Flask is designed to be easy to use. Each path is handled by a different method below is part of a basic example showcasing this.

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "GET":
        return render_template("login.html")
    elif request.method == "POST":
```

`@app.route()` allows us to list which path should call the function below and which methods the request can use. We can then look at `request.method` to have the function perform different actions depending on the method used by the request. Here if we use the GET method on the `/login` path, then the function is designed to just return the `login.html` page and tell the browser to display it. GET requests are typically used when a browser wants to receive data from the server such as getting the HTML, CSS and JavaScript for a login page.

Below is an example of a redirect. This function will, no matter the method used for `/login.html`, redirect the request to the login function shown earlier. This is done to ensure the user does not have a `.html` on the login page, thus allowing for a cleaner URL.

```
@app.route("/login.html")
def login_redirect():
    return redirect(url_for('login'))
```

POST requests are typically used when a browser wants to send data to or create/update a resource. An example of a POST request would be when a developer or admin wishes to edit the 'question' of a mission. Where the browser will send Flask the id of the mission to be updated, and the updated question. Wherein Flask will attempt to parse that data, and then communicate with the database to update the mission.

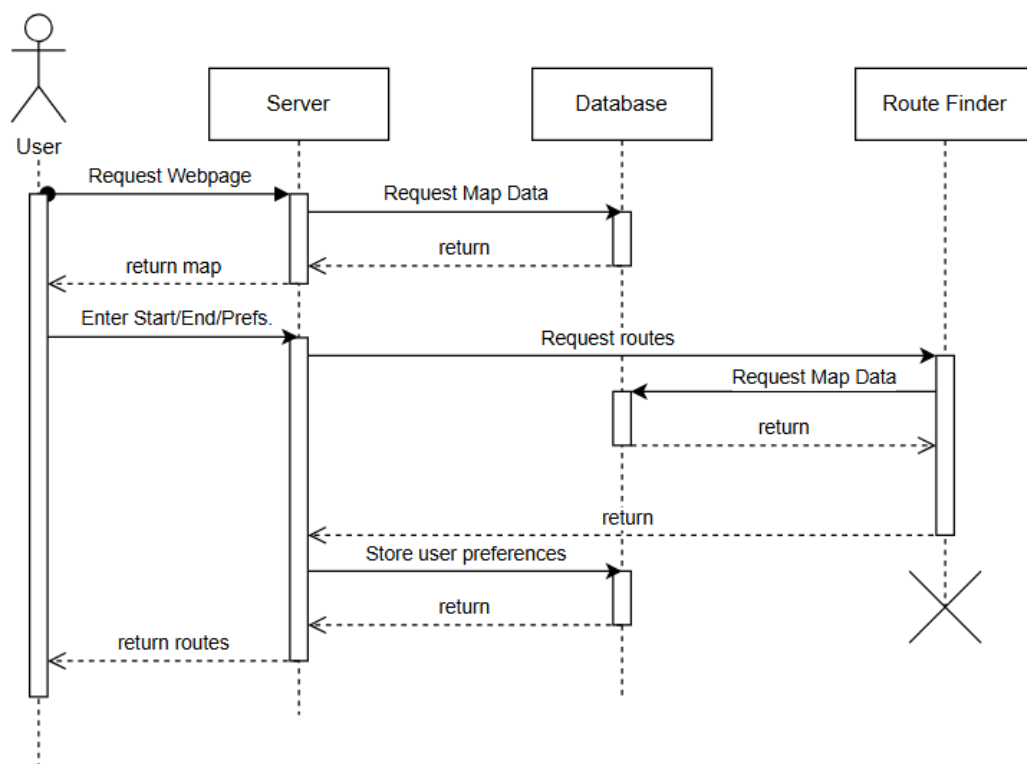
```

try:
    data = request.get_json()
    id = data["id"]
    question = data["question"]
except:
    id = None
    question = None

```

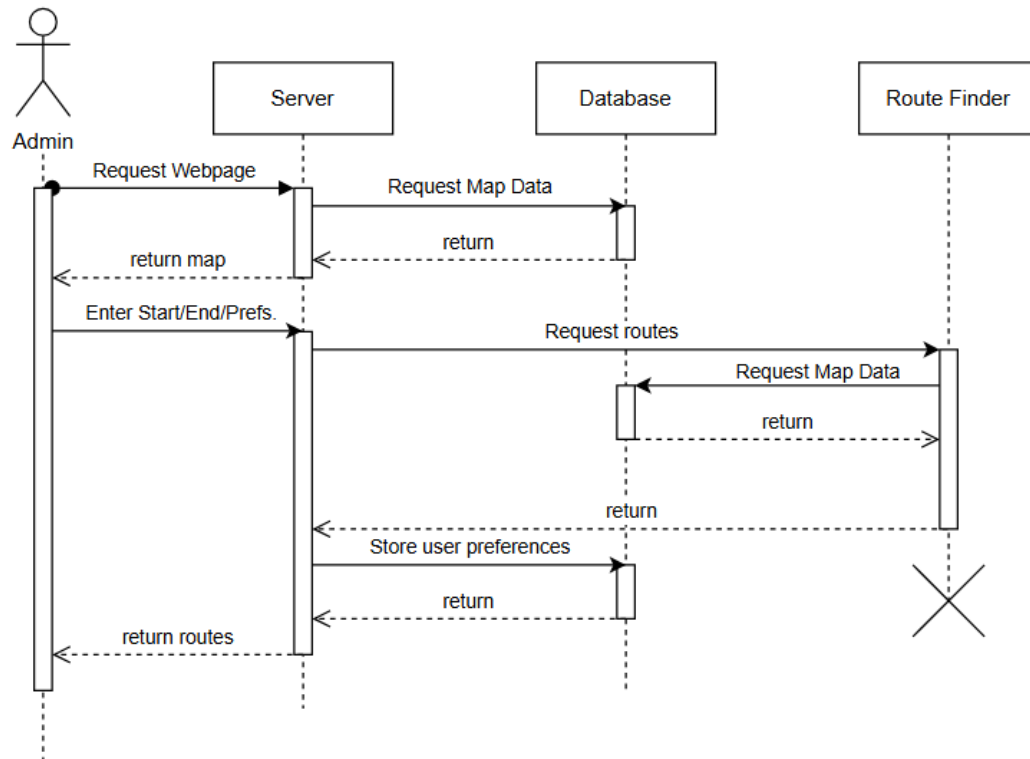
Front End and Back End Interaction

We created diagrams to show the flow of the program logic when different roles are using the application. In the following diagrams, we follow the perspective of either a user or admin and the different actions they can take when navigating the user interface. These diagrams were essential when it came to helping us understand how each component of the application works with each other.

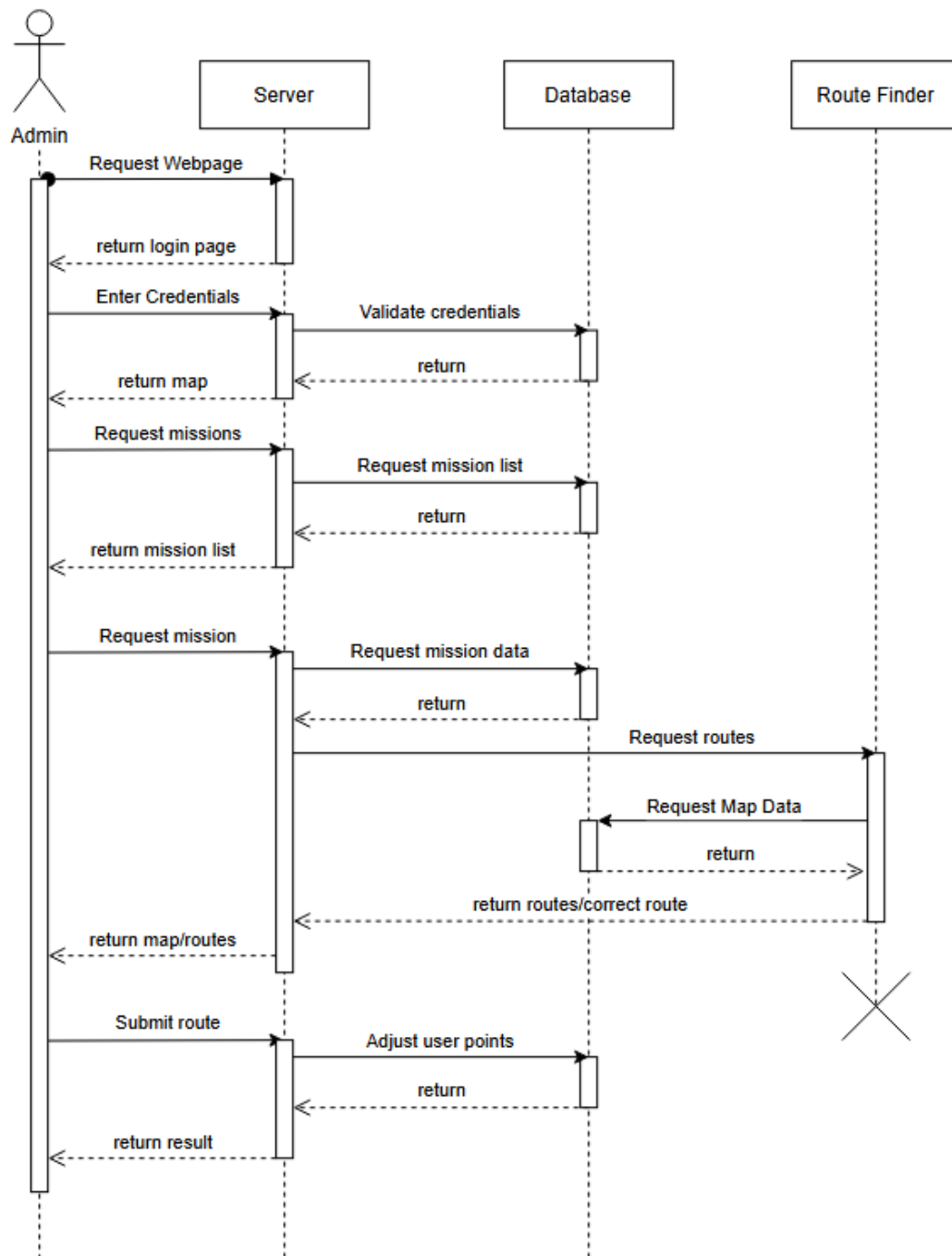


The diagram above shows the flow of the application from the perspective of the average user. Here, we can see that the user requests the webpage from the server, which then requests the map data from the database (e.g. nodes, segments). The database returns this data to the backend server, which then returns the map to the front end with all road data. The user can then select their start and end points on

the interactive map, which is sent to the backend for processing. The backend then sends this data to the routefinding algorithm and requests the optimal routes. The route finder accesses map data from the database to be used in its calculations. It then computes the optimal routes and returns them to the server. The server returns these routes to the frontend, where the UI visualises them for the user. Hence, we have fulfilled our user's most essential need; to select two points and receive the most optimal routes between them.



The diagram above shows the abilities of an admin using the application to plan routes. This behaves in the exact same way as the previous diagram, however, it is important to clarify that the admin is able to do everything the user can still do. The next diagram will display the additional actions the admin can take.



The diagram above demonstrates the flow of logic through the application when an admin logs into the system and accesses the mission data. When logging in, the admin enters their credentials via the login page which are then sent to the backend server. A database query is made on the backend to verify the identity of the admin. If the credentials are valid, the map is returned to the frontend and the admin has access to the main application. If they wish to see the missions, they can request this on the user interface and the backend will query the database for mission data and return it to the frontend to be displayed on the user interface. If the admin wants to request data regarding a specific mission from that list, the locations revolving

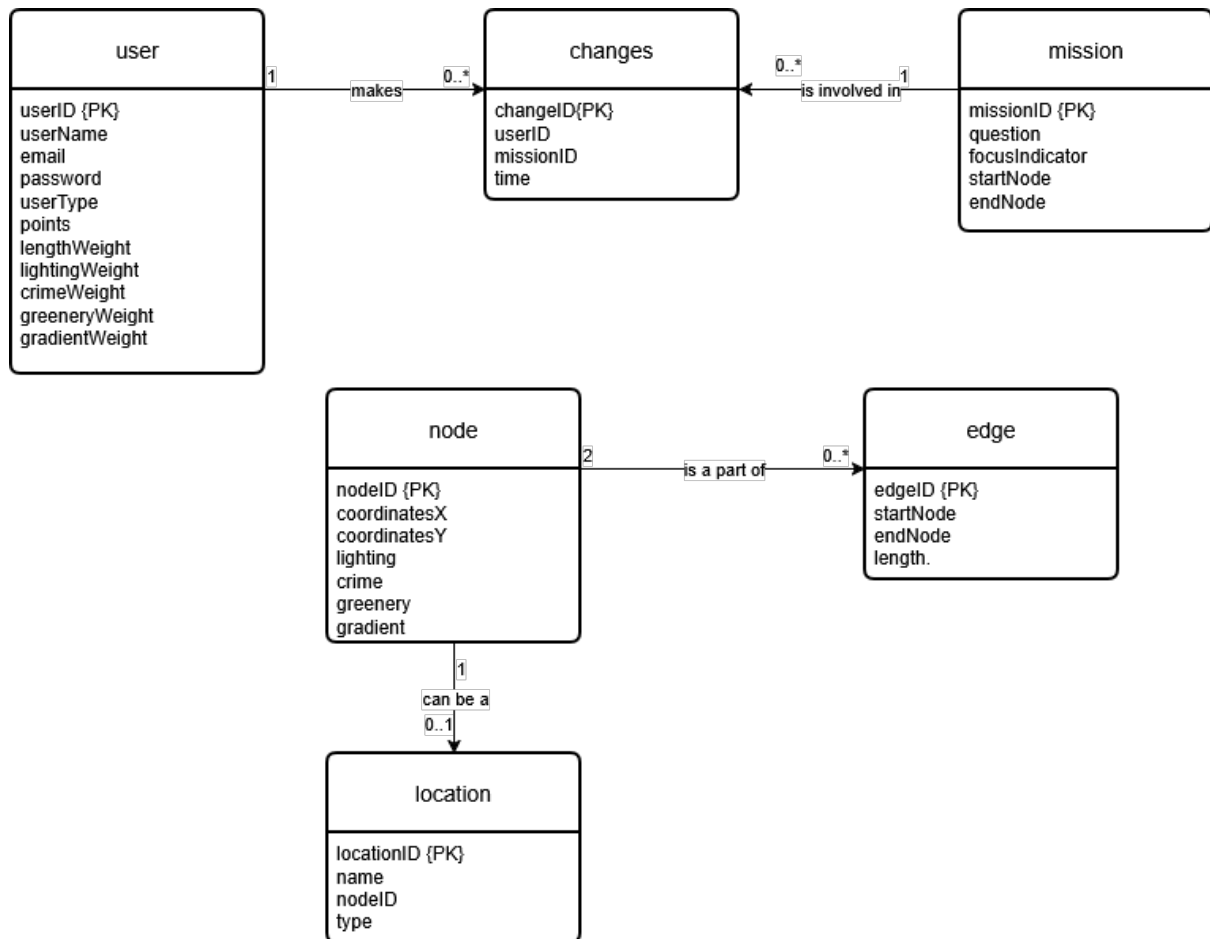
around that mission are identified from the database and their corresponding routes are computed by the routefinding algorithm and returned to the frontend to be displayed. This structure helped our frontend and backend developers successfully build the functionality of the HTTP request manager using Flask.

Data Collection

To collect the data we wrote a script that would access the Opensstreetmaps database and collect a medium resolution dataset of all road nodes and edges within 1.5km from the city centre of Exeter. To do this we: used the osmnx package for python to get a node graph of the road network 1.5km around the point 50.7184 , -3.5339, which are the rough global coordinates for Exeter city centre. Then we added new node IDs to the nodes, starting from 0, so they could more easily be managed by our route finding algorithm. Then we removed all duplicate edges e.g. node one links to node 2, and node 2 links to node 1, as our algorithm only needs one link. And then we exported the node and edge tables in a .db file with empty tables for locations, users, changes and missions to be filed later.

Database

To plan the structure of the database that stores user and road information, we designed the following entity relationship diagram to identify the key entities in our project for sprint 1, their attributes and relationships. It follows the UML notation.



The data is stored in a .db file containing 6 tables (users, changes, missions, nodes, edges, locations). This data is accessed and modified by the application by using the DatabaseMethods object's methods which use the sqlite3 python library to interact with the .db file.

Users:

The user table is used to store data about the user and their preferences. The attributes userName, email and password are used when logging in. When the user inputs their details into the login screen, the method getLoginDetails() can be called which searches for the username and email combination and returns the corresponding password. The returned password can be compared to the user's input to validate their identity.

The userType attribute stores whether the user is a "Traveller", "Administrator" or "Maintainer". The map can call the getUserType() method when a user attempts to make an edit to the map (e.g. adding a new node) and check if the user is an admin or maintainer before calling the actual edit methods and sending the edits to the database.

The points attribute is used to fulfill the requirement “Users earn points/badges and can track progress” for route quests. There is a corresponding addPoints() method which can be called after a user completes a mission to increase their score.

Finally, there are user weights for all the indicators. They can be modified and retrieved using the setUserWeights() and getUserWeights() methods. These are used by the routing algorithm when working out the weights for its weighted graph.

Changes:

The changes table is used to store who makes changes to the missions, when they made the change and which mission they changed. The attributes in this table are foreign keys representing the related user, mission and the time stored as an integer (the number of seconds since epoch). This is added to whenever an admin edits a mission using the editMission() method. It can be retrieved from the database using the getLog() method.

Missions:

This table is used to store data about the missions in the “Route Quest game mode” required for the project. Aside from its main primary key, it stores the attributes question (which is text which describes the objectives of the missions e.g. “find the route with the best lighting between x and y”), focusIndicator (which contains the primary indicator being asked about. In the question example given earlier it would be lighting) and startNode and endNode (which contain the beginning and end of the mission routes respectively).

There are multiple methods used for interacting with mission data. Firstly, there are methods addMission() and editMission() which are used by administrators to modify the mission table in the database (editMission() is also able to add a change to the changes table whenever it is called, fulfilling the requirement for a changelog). There is method getMissionSelectData() which returns a list of mission questions and their IDs which can be used to display a menu for the user to select a mission. getMissionData() can then be called using the id selected by the user and returns necessary data for actually running the mission (the focusIndicator and start and end nodes). Finally, once the user completes the mission by selecting a route, they are given feedback in the form of points. If the user selected the correct route, the addPoints() method can be called to increase the user’s points attribute.

Nodes, locations and edges:

Nodes contain the latitude and longitude of every node (attributes coordinatesX and coordinatesY) as well as its indicator values (lighting, crime, greenery, gradient).

Locations contain a foreign key reference to their node as well as the name of the location and its location type (e.g. park, station, school). Edges contain a foreign key reference to the two nodes involved in the edge as well as the length between them. Nodes and edges are used by both the map and pathing algorithm. Locations are specifically used by the map.

The methods `addNode()`, `editIndicators()`, `addLocation()`, `addEdge()` and `deleteNode()` are used whenever an admin wants to edit the map and add those changes to the database.

`getMapData()` and `getLocationList()` are used by the map to display the graph to the user on the map.

Finally, `getAllNodes()`, `getAllEdges()` and `getSurroundingLength()` are used by the pathfinding algorithm to get information about the graph in order to calculate the best routes.

Route Finding Algorithm:

The route finding algorithm uses the SciPy implementation of Dijkstra's algorithm to find the best path between two points according to the user's weight preferences. The route finding algorithm originally used my own implementation of Dijkstra's however this was inefficient since it used python lists and dictionaries so I switched to the SciPy implementation since this used NumPy 2D arrays which made it a lot more efficient. It then finds the two other alternative routes by changing the weighting preference so that the algorithm will give a different route that would be better if the user had those preferences. If this fails to generate 3 routes because there is not an alternative route that is suitable it will remove segments from the segment list so that the algorithm is forced to pick an alternative path.

The `FindMultipleRoutes` first calculates the total magnitude of the weightings and divides each weight by this amount so that each of the weights are always below or equal to one, the method then finds n routes between the selected start and end node. It first gets the segments, nodes, and user weightings from the database, then gets the first route using the `findroute` method and the `getpath` method to get the actual path that the route follows.

It then calculates the seed from the start and end nodes so that when the same start and end nodes are selected the same routes will be generated.

It finally finds the rest of the paths by running `findOtherRoutes` until it gives enough routes.

The `findOtherRoutes` method randomises the user weights to find other routes that may be optimal when other user weights are used, however are not the correct route given the actual weights given.

It first calculates a random value for each of the weights.

If the method is set to remove nodes it will pick a segment out of each route already generated and then remove every segment connected to that node from the list of segments.

The method then finds another route using these updated weights and segments; If this route is different to all previously generated routes it returns the route, if it is not it will try again with new weights and remove another set of segments (if the method is told to remove segments in the input).

The findRoute method calculates the actual route when given the segments, nodes, start location, end location and the weightings. If weightings are given it will go through each weight in and multiply it by the indicator, the weighting cubed and the length it then sums this total and that is the total weight of a segment. If no weight is given the algorithm will take the length as the weight.

It then adds this weight to a list of segments and does this for all the segments. It then puts these segments into a numpy array and uses SciPy's implementation of Dijkstra's algorithm to find the shortest path between the nodes given the selected weights.

It then returns the distance matrix and the predecessor matrix.

The getPath method returns the path between the start and end nodes given a predecessor matrix. It does this by iteratively selecting the next node in the path until it reaches a value of -9999 which indicates it has gotten to the start node of the route.

Variable Name	Description	Data Type	File
segments	Stores the segments given by the database	list of tuples	routefindingalgorithm.py
nodes	Stores the nodes given by the database	list of tuples	routefindingalgorithm.py
whereRouting	Which node the script is routing to and from	tuple	routefindingalgorithm.py
weightings	Which weightings the algorithm should prioritise	tuple	routefindingalgorithm.py
weightedSegments	The segments after the weightings have been used to make a single final weight	list of tuple	routefindingalgorithm.py
segment	A single segment between 2 nodes	tuple	routefindingalgorithm.py

start	Start node of a segment	int	routefindingalgorithm.py
end	End node of a segment	int	routefindingalgorithm.py
length	Length of a segment	float	routefindingalgorithm.py
weight	Weight of a segment	float	routefindingalgorithm.py
node	A single node	tuple	routefindingalgorithm.py
tempWeight	A single weight from weights	tuple	routefindingalgorithm.py
distmatrixSize	Size that the distance matrix will be	int	routefindingalgorithm.py
distMatrix	Dist matrix for segments to be put in	2d array	routefindingalgorithm.py
distances	Weight between each pair of node	2d array	routefindingalgorithm.py
pred	Predicator matrix	2d array	routefindingalgorithm.py
seed	Used to make the same route generate when the same 2 nodes are given to start and end at	int	routefindingalgorithm.py
similarityNeeded	How similar the routes can be to be accepted	float	routefindingalgorithm.py
removingNodes	If the algorithm should remove nodes to generate more routes	boolean	routefindingalgorithm.py
weightingsMagnitude	Total of all the weights	float	routefindingalgorithm.py
escapeCounter	Used to make sure the algorithm doesn't run forever when enough routes cannot be found	int	routefindingalgorithm.py
singleRoute	A single route between the 2 nodes	list	routefindingalgorithm.py
exitready	If all segments that lead to the node have been removed	boolean	routefindingalgorithm.py
routeWeights	The weights between each node	2d array	routefindingalgorithm.py
routePr	Predicator matrix	2d array	routefindingalgorithm.py
route	Route between start and end node	list	routefindingalgorithm.py
firstRoute	The first route that it generates	list	routefindingalgorithm.py
isDifferent	If the routes are different enough	boolean	routefindingalgorithm.py

Similarity	How similar the routes are	float	routefindingalgorithm.py
userID	The users ID	int	routefindingalgorithm.py
numberOfRoutes	How many routes to generate	int	routefindingalgorithm.py
myDatabase	The database object	object	routefindingalgorithm.py
weightingstemp	Temporary weightings before they are scaled	tuple	routefindingalgorithm.py
tempWeight	A single weight	float	routefindingalgorithm.py
largestWeight	The largest of the user weights	float	routefindingalgorithm.py
weightIterator	Used to iterate through the weights	int	routefindingalgorithm.py
firstRouteWeights	Matric of weights between every pair of nodes	2d array	routefindingalgorithm.py
actualRoute	The route between the start and end node	list	routefindingalgorithm.py
newRoute	The next route generated	list	routefindingalgorithm.py
tempSegments	The segments that will be changed when a node is removed	List of tuples	routefindingalgorithm.py
path	The path between nodes i and j	list	routefindingalgorithm.py
Pr	The predicator matrix	2d array	routefindingalgorithm.py
i	The start node of a route	int	routefindingalgorithm.py
j	The end node of a route	int	routefindingalgorithm.py

Map design and implementation

The map was built using the open-source Javascript library Leaflet.js. Not only does it provide excellent, easy to use methods for all our relevant mapping purposes, and has a well-written and greatly laid out documentation, but it also allows for mobile platforms to interact just as well as desktop platforms on the web application. For these reasons, we chose the Leaflet API to create our map and used Javascript with HTML elements for the map backend logic and to provide user interaction.

Although our map's index.html file contains many methods, the key functions and logic should be explained. Excluding route finding, there are three main categories of functions that perform the following logic: creating and saving data, fetching data from the database, and editing data. In this context, data refers to our map's nodes, edges, and locations. There is a sequential set of steps from when the user loads the map that allows for map states to be saved and fetched from the database so that a user can truly make lasting changes to the map.

When the user loads the map, it begins with nothing but the Open Street Map provided by Leaflet. The following set of functions run.

```
////////// ON LOAD FUNCTIONS //////////

async function loadMapData() {
  const res = await fetch("/getmapdata");
  const data = await res.json();
  if (data.nodes.length === 0 && data.edges.length === 0 && data.locations.length === 0) return;
  renderMapFromData(data.nodes, data.edges, data.locations);
}
loadMapData();

function renderMapFromData(dbNodes, dbEdges, dbLocations) {
  dbLocations.forEach(createLocationFromData);
  dbNodes.forEach(createNodeFromData);
  dbEdges.forEach(createEdgeFromData);
}
```

The route /getmapdata activates the getMapData() function inside database_methods.py which returns the backend index.html all the useful information about nodes, edges, and locations to build the map from what is already present in the database. ForEach() functions are run for each node, edge, and location entry fetched from the database.

First, the locations are created but not yet displayed for the client. Locations and nodes use different icons and this order allows for when nodes are chosen to be displayed, it can be checked whether a location already exists with the given node ID and a location icon can be used instead. With the location ID, node ID for the given location, name, and type, a new Location object is generated and added to the locations array, locationsMap Map with location ID for key, and the locationsByNodeID Map with the node ID for key. With this, the client is ready to perform adding nodes and edges next.

Nodes are similarly created, this time with Node-specific attributes such as the node's coordinates in latitude and longitude, and the indicator values associated with the node. In the backend, Nodes objects also contain a connections attribute which is an array that stores the node IDs that the node connects to. This is not present in the database, and is client-side only, because only the client needs to know each

node's connections for ease of viewing and editing by an admin. These connections are easily built from the Edges table in the database when edges are created. Each Node object is added to the nodesMap Map with node ID as key, and nodes array. In this same function, createNodeFromData(), Leaflet markers are created and assigned their popup contents based on whether the node ID being used to build the current node already exists in locationNodeById. If it does, then a location popup should be used. Otherwise, a simple node popup. Regardless of the choice, the markersMap Map is added an entry of the marker with the node's ID for key, used for editing the popup if a user decides to make a change to a node.

Finally, edges are created for the client in the same fashion. Segments (edges) are much simpler data structures, containing only the attributes of ID, fromID, and toID, which represent the logical connection from where the edge originates and to where it goes. First, we check if the node the current edge connects from already exists on the map. If so, then logically, wherever the edge ends with endNode, is also a connection for that node. So, for the given starting node, we push the endNode to its connections attribute. This happens for every edge that happens to originate from a node already on the map, thus filling out the empty connections arrays we set up earlier. Also, the markersMap is used to access the nodes' popups and update them with the correct new value for connections. The array edges is pushed the new Segment object. No Map object is needed to store Segments as edges are directly influenced by interactions with nodes.

The local variables for nodeId, edgeId, and locationId are incremented by one each time a new object is created, added onto the ID value fetched from the database. This way, if there are missing entries in the database, the IDs stored in the client are always the most up to date with the database.

With all the load functions performed, the user is presented with an admin's view of the map having all nodes and locations present and up to date with the database. Now, if the admin would like to add new nodes or locations, the database is ready to be written to with information from the client. Leaflet's map.on('click') event registers the location of left click and creates a temporary node creation popup. The admin can enter where the node connects to inside an input field, and upon clicking Save, the saveNode() function is run with the latitude and longitude of the click event given as parameters. Now, the connections input is checked to fit the regex requirements of no leading zeroes and positive numbers only and verified to be present in the input field, separated by commas. Duplicate connections values are also removed. For each number in the connections input field, the function createSegment() is run which creates the necessary edges client-side containing the ID of the node being created for startNode and the connection number for endNode. Multiple edges may be created for one node, as many as there are separate connections. POST requests are sent to the server for edge creation in the database. This is relatively straightforward, and the same happens for when the Node is then added to the

database as the `addNode()` and `addEdge()` functions `database_methods.py` make this simple. However, logically, an issue arises when a node is created with connections for nodes that do not exist yet.

Because the database uses foreign keys that we enforce for integrity, the database cannot create an edge with an `endNode` attribute for a node that doesn't exist yet. The same happens for the `startNode` entry, because the POST request to create an edge happens before the POST request to create a node. Thus, an edge being created will have no node entry to refer to for `startNode`, but may have an existing entry for `endNode`. So, a check is made to see if both `startNode` and `endNode` are in the database already inside the `Nodes` column. If they are not, then a placeholder node is created, a hollow node, possessing only the ID of the `endNode` or `startNode` required, so that the edge can have a valid node ID to reference for `startNode` and `endNode`. When a node needs to then be added to the database via POST request to the server, the server checks if the node ID already exists in the `Nodes` table and if it does, it means it is a placeholder node and the function `updateNode()` is run in `database_methods.py` instead of `addNode()`. At the end, we are left with new entries for valid nodes and edges in the database, which can then be loaded by the client when refreshing or re-accessing the webpage.

The same process is followed for `Locations` when right click is pressed. Leaflet allows the detection of right click and its position on the map with `map.on('contextmenu')`, and the function `saveLocation()` is run with the latitude and longitude of the click. `Locations` and nodes are separate, though interlinked. Every `Location` is also a node, but not every node is a `Location`. So, locations act like 'armour' to a node, by referencing a real Node ID, and providing a couple more details and utility than a regular node, including a designated name, type which will be implemented inside Sprint 2 for displaying different icons or categorisation of locations, a dedicated location ID, and a bigger red popup icon to show a node is a `Location`. `Locations` do not contain coordinates or connections as these are stored inside a location's 'complementary' node which the `Location` references inside the database. `Locations` are likewise added to the database with a POST request through the `/addlocation` route, and the location's complementary node (the actual body of the location) is also created and added to the database, allowing for effective saving of a new location.

Lastly, node editing is a crucial admin feature. In this context, node refers to locations as well because they have the same editing features. Once created, the admin can click on the node's marker and choose to edit its indicator values which are automatically generated as 0.0 for faster and less clunkier initial node creation. Any value from 0 to 1 is accepted for indicators. The connections attribute of the node can also be edited, however it cannot be left blank. Upon clicking save, `editNode()` is run which updates the node's connections client-side, uses these changes to update the edges both client-side and database-side, and likewise edit the indicators client-side and database-side. All the edges starting from the node

being edited are deleted and re-created with the new connections provided by the admin to avoid the need to calculate which edges (connections) were changed and which were not. This does not influence the speed of the edit in any noticeable way as such a small amount of changes are being worked with. Finally, markersMap is used to access the marker for the given node being edited and change its popup to reflect the new changes.

Testing

Unit Tests

Some aspects of the program can have their functionality ensured by rigorous unit testing. Because of the small number of internal programs and most of the system is contingent on user interaction, not much of the program can be unit tested.

Unit tests cover the Database Methods program as follows:

Test Number	Test Name	Intention	Outcome
1	testAddUser	Test the creation of users in the DB	Pass
2	testUserWeights	Test the initialisation and saving of user preferences in the DB	Pass
3	testAddPoints	Test the tracking of user points from missions in the DB	Pass
4	testDeleteUser	Test the deletion of a user from the DB	Pass
5	testGetNodes	Test the storage and initialisation of location nodes in the DB	Pass
6	testUpdateNode	Test the updating of location nodes in the DB	Pass
7	testGetAllEdges	Test the storage and initialisation of edges in the DB	Pass
8	testUpdateEdgeLength	Test the updating of edge lengths in the DB	Pass
9	testGetSurroundingLength	Test the foreign key functionality of nodes and edges in the DB	Pass

10	testDeleteNode	Test the deletion of nodes and ensure clean deletion which supports the foreign key constraints of the DB	Pass
11	testGetLocations	Test the saving, initialisation and collection of named locations from the DB	Pass
12	testGetMissionData	Test the storage and retrieval of mission data in the DB	Pass
13	testEditMission	Test updating a mission in the DB	Pass

Since all our unit tests pass we can be sure that the functionality of the individual components is robust and should not become problematic when we integrate them with user input. It was absolutely integral that the database handler passed all unit tests since the entire application requires reliable data at multiple stages.

To run the unit tests for yourself, navigate to the testing folder in the source code. Run [databaseTests.py](#) whilst in that folder.

End to End Manual Tests

The manual testing for the project follows a similar structure to our demonstration in our presentation. We test the functionality of the user interface and how well it communicates with the backend of the server. To ensure good code coverage, interactions are comprehensively tested based on what URLs we serve to the user. Files covered by these tests are [server.py](#), [database_methods.py](#), [routefinding.py](#) and all of our html templates.

Since our security system is not fully implemented yet, by default the user is signed into an admin level account. Admin level accounts can still use the route-finding features as usual, as well as making changes. This is useful for the purpose of testing since we can see how admin changes affect route-finding and user experience.

Route-finding and Map Tests

First we start the server by running [server.py](#) and then navigate to <http://127.0.0.1:5000/> . We should now see the map/home screen.

Custom Routes in Exeter, UK

This tool does not reflect real world data

This tool is not suitable for emergency use

Missions

Route Finder

Start Node: 300 End Node: 600 Find Route Clear

Start Location: The Imperial End Node: Southernhay Find Route Clear Distance: 1 Lighting: 0 Greenery: 0 Gradient: 0 Crime: 0

Enter start and end nodes and click "Find Route"

Hide Nodes



Route Finding by Node ID

The first option we are given is to input our start and end nodes by using their numeric IDs, we can find these IDs by clicking on the node markers. Pressing find route will generate 3 separate routes after a short delay, these are given as a list of nodes along with their distance and a breakdown of indicator scores along the route. Colour coded routes are then drawn on the map. From these screenshots we can verify this stage of the test passes.

Route Finder

Start Node: 300 End Node: 600 Find Route Clear

Start Location: The Imperial End Node: Southernhay Find Route Clear Distance: 1 Lighting: 0 Greenery: 0 Gradient: 0 Crime: 0

Route from Node 300 to 600:

Red Path: 300 → 6 → 7 → 132 → 675 → 153 → 8 → 671 → 9 → 678 → 676 → 31 → 95 → 657 → 330 → 178 → 176 → 174 → 30 → 97 → 591 → 808 → 809 → 10 → 810 → 335 → 270 → 102 → 545 → 101 → 169 → 272 → 262 → 273 → 654 → 534 → 100 → 540 → 459 → 461 → 430 → 339 → 340 → 602 → 600

Total Distance: 2793.00

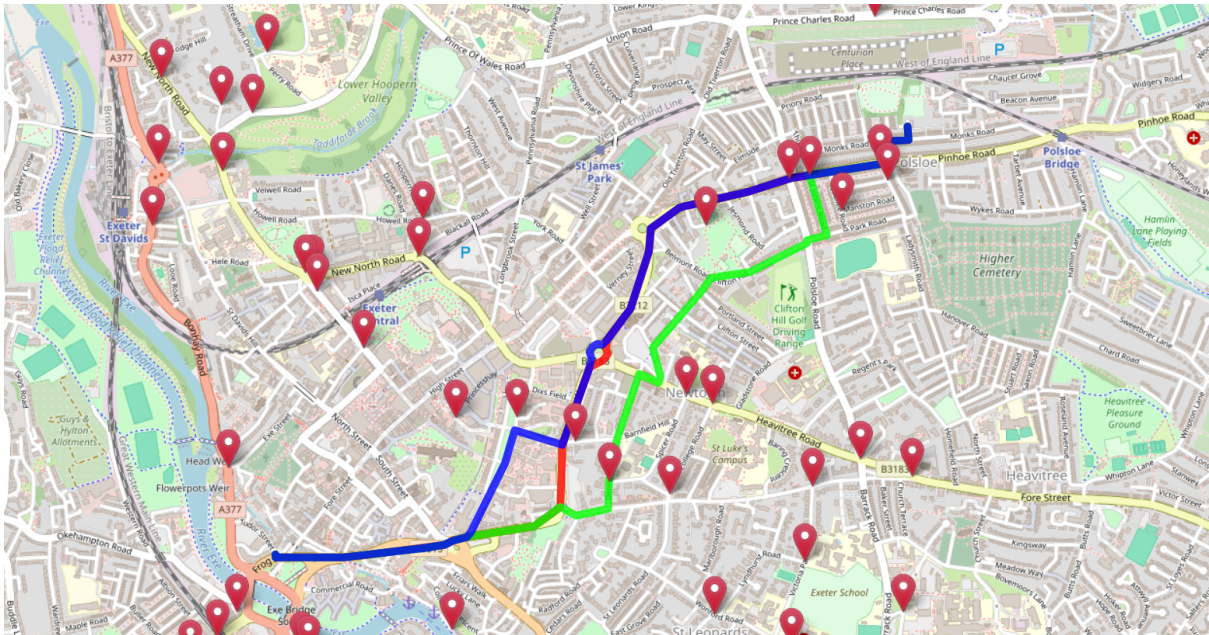
Score Breakdown: Lighting: 0.46Crime: 0.31Greenery: 0.48Gradient: 0.53

Green Path: 300 → 6 → 7 → 132 → 675 → 153 → 8 → 671 → 9 → 678 → 676 → 31 → 95 → 657 → 330 → 178 → 176 → 174 → 30 → 65 → 845 → 26 → 34 → 39 → 826 → 33 → 27 → 202 → 761 → 710 → 760 → 267 → 604 → 370 → 105 → 277 → 275 → 458 → 460 → 459 → 461 → 430 → 339 → 340 → 602 → 600

Total Distance: 2965.00

Score Breakdown: Lighting: 0.46Crime: 0.33Greenery: 0.46Gradient: 0.48

Blue Path: 300 → 6 → 7 → 132 → 675 → 153 → 8 → 671 → 9 → 678 → 676 → 31 → 95 → 648 → 99 → 679 → 327 → 96 → 321 → 97 → 591 →



Pass

Route Finding by Location Name

After clearing the map screen using the clear button. We go through the same process as above but enter a pair of named locations. These names can also be copied from clicking on the location nodes. This also works.



Pass

Route Finding with User Preferences

To test this we change the preference fields to some different values, here, prioritising Lighting and Greenery. Then we run the route-finding query again.

Route Finder

Start Node: End Node: Find Route Clear
 Start Location: End Node: Find Route Clear Distance: Lighting: Greenery: Gradient: Crime:

We can see that this generates different routes to the preferences we had before,

therefore the test passes and we know that our user-preference selection works with the weighted map.



Pass

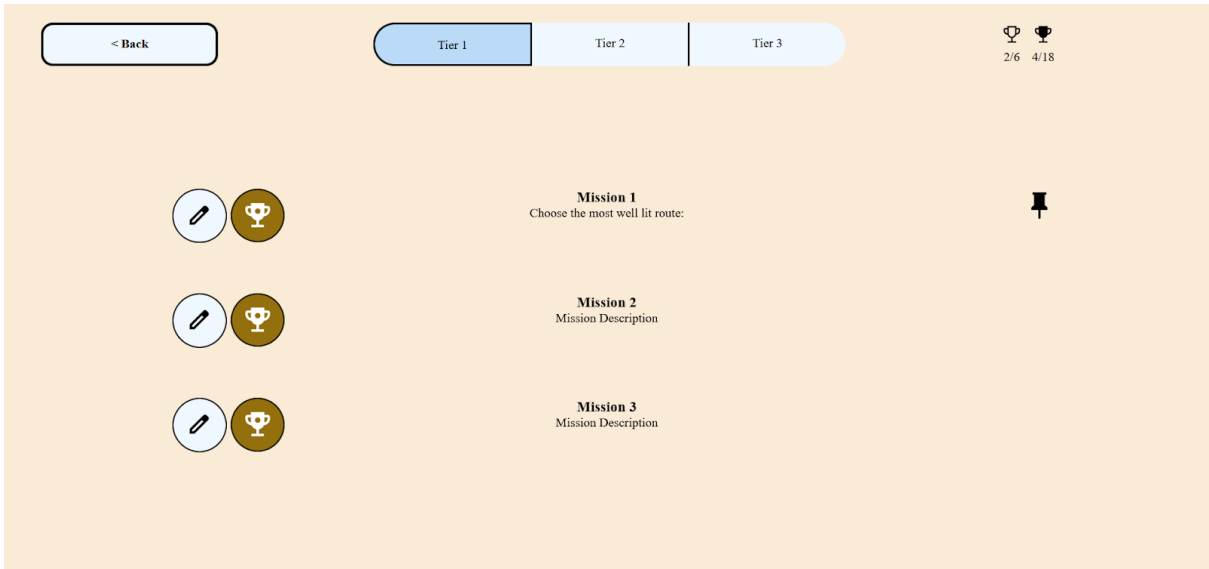
Mission Tests

Clicking on the mission button at the top of the page underneath the disclaimer will take us to the mission screen, as shown below.

Custom Routes in Exeter, UK
This tool does not reflect real world data
This tool is not suitable for emergency use
Crime and Lighting indicators are synthetic and do not reflect real world values

Missions

Route Finder



Selecting and Completing a Mission

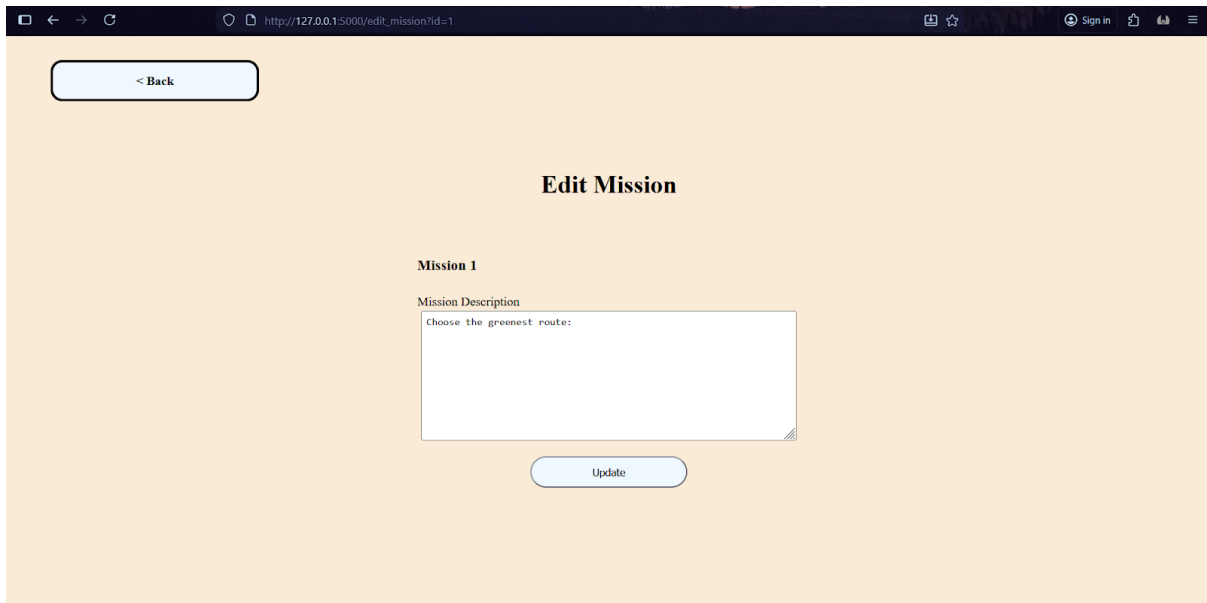
Next we click on Mission 1. Choosing either Blue or Green will show the user “incorrect”. Choosing red (the correct route) will give the user a “correct” pop-up and redirect them back to the mission screen.



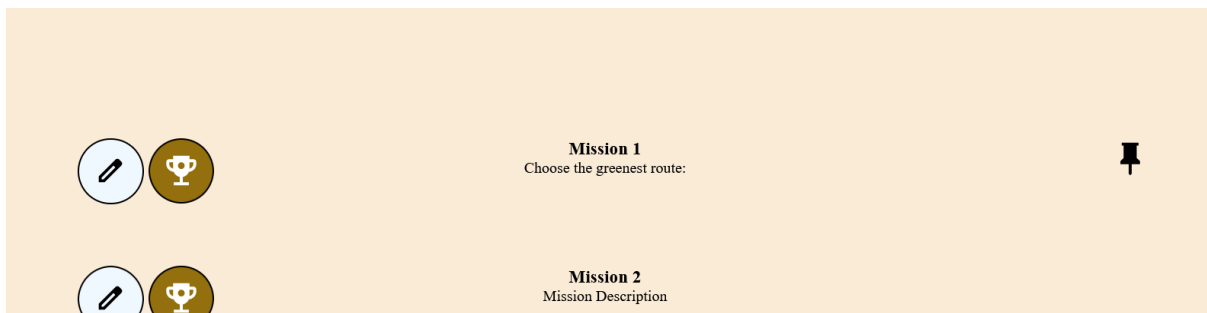
Pass

Editing Missions

Clicking on the pencil icon next to the mission will open the mission edit screen.



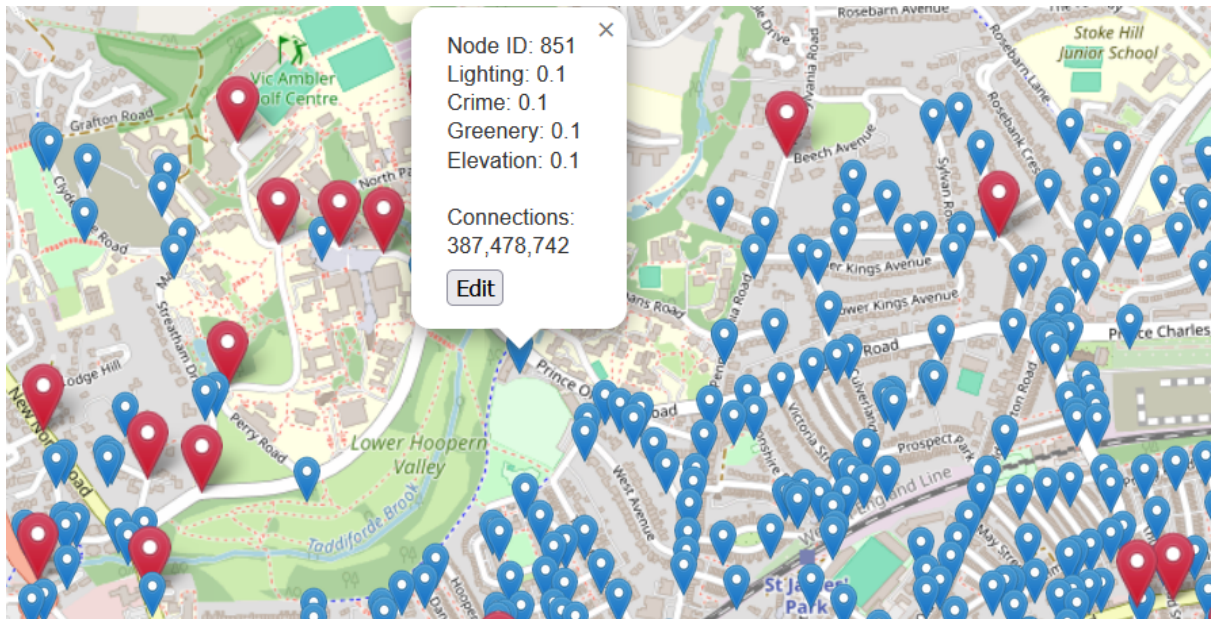
Change the text in the box and then click update. This change should now be reflected in the mission list.



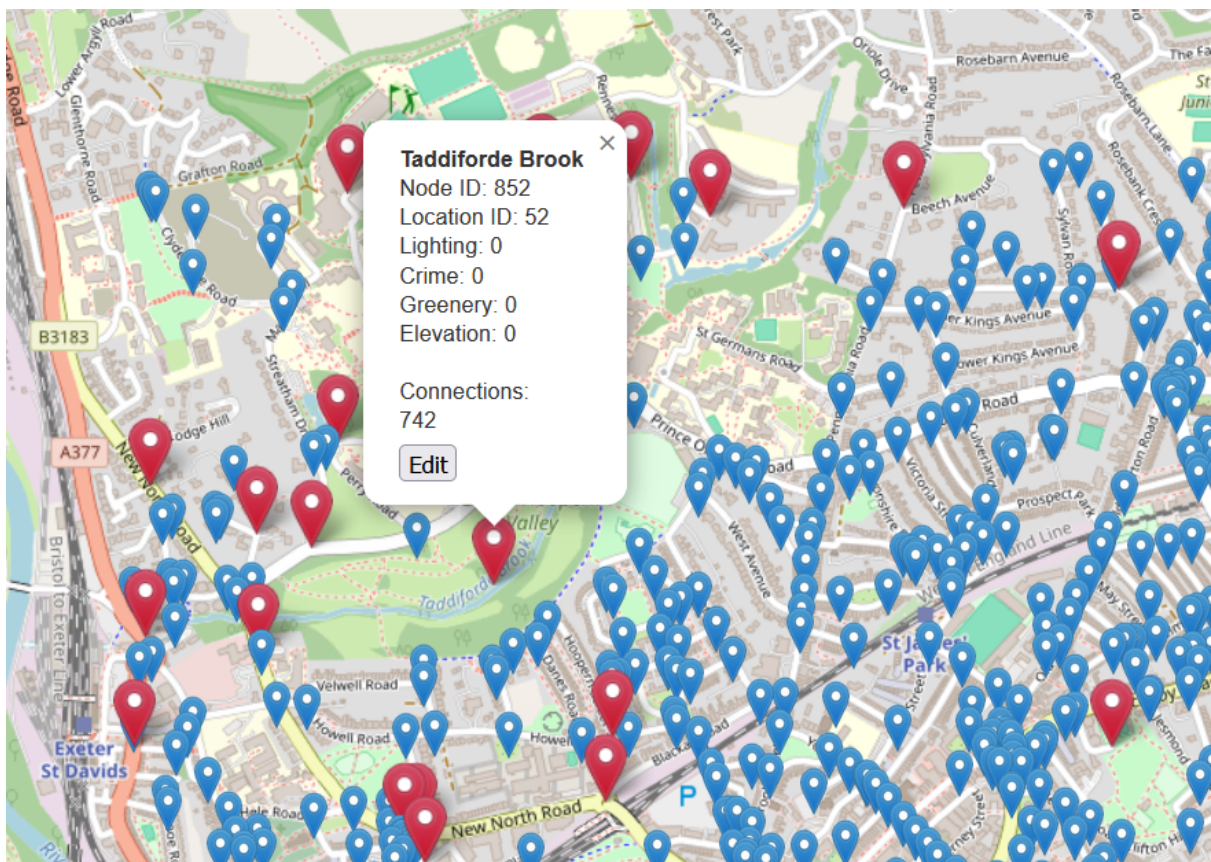
Pass

Adding/Editing Nodes - Admin Features

Pressing the back button from the mission screen will take us back to the map screen. Here we are able to test some of the admin features. Left clicking on the map will create a new node, we can then add connections to the node. We can also edit the indicator values on the node. Restart the server and we can see the changes persist.



Right clicking on the map will open the Location Creation pop-up. This will also be saved on server reload.



Evaluation

Our evaluation will adapt over the course of sprint 1 and sprint 2, and should deliver a full critique of our project by its completion. In its current state, there is still much more work to be done in sprint 2 before the project reaches the bounds of the final scope. However, in this section we will evaluate our performance for sprint 1 and plan ahead to a streamlined sprint 2 by identifying flaws in our current approach and

changing them for the future. We will do this by referring back to the success criteria we designed in the problems and requirements section, taking each measure individually and discussing how well it has been achieved. Through this discussion, we will outline why we have failed where we have, and what this teaches us about possible risks in the second sprint.

Program Functionality

Overall, the basic functionality of our program has been completed, and the general aim of this sprint has hence been fulfilled. Our program works as intended, and we have mostly fulfilled every requirement we laid out for the first prototype. However, there are a couple of exceptions to this. The table below is the sprint 1 acceptance criteria devised in the problem identification section with an additional column to visualise what requirements have been met (green) , partially met (amber), or not met at all (red).

Requirement Number	Requirement Description	Measurement Method	Requirement Completion
1	Users can select two locations on an interactive map	Functional Test: start and end points can be selected without errors.	
2	Compute at least 3 route alternatives between two points	Automated/Manual test verifying that ≥ 3 unique routes are returned.	
3	Compute a route scoring using at least 5 indicator values	Automated/Manual test verifying that a route score is returned when two locations are selected. Code inspection should verify that at least 5 indicator values are used in scoring function.	
8	Route segments should be displayed on the map	Visual inspection (screenshot) of interactive map rendering routes.	
10	Admins must be able to create/edit missions	Functional test confirming that	

		missions can be created/edited successfully.	
14	Mobile-friendly layout	Visual inspection of the application on a mobile interface, ensuring it is still functional.	
15	User can complete preplanned "missions" between points assigned to them	Test confirming that users can complete missions.	

As evidenced in the table above, we have fully met almost every requirement we sought out to complete in this sprint. We have not met the requirement of a mobile-friendly layout, and this needs to overflow into sprint 2. We also have not yet finished the requirement that states an admin must be able to create or edit missions. In the current state of the application, an admin can edit missions but cannot create new ones. Both of these requirements will be added to the sprint 2 acceptance criteria for eventual completion. We believe that we outlined a scope that was slightly too large for our first prototype, and these requirements suffered because of that. The mobile-friendly layout does not relate to the basic functionality of the application, and it was too ambitious to try and group it in with the rest of sprint 1 development. It is more suited towards sprint 2, which is supposed to expand upon pre-existing features. We lost development time in designing the mobile-friendly layout wireframes instead of adapting the pre-existing wireframes into a mobile application later down the line. However, it could be argued that our dedication to designing all wireframes at once helped keep our vision consistent and it will save time in sprint 2. When the development of one feature lags behind, it can affect the development of other features too. In this case, the admin mission creation system was affected by our focus on other features that were taking longer than anticipated. We also put a lot of time towards a user login system, which was not required in sprint 1. This was another key contributor to our scope creep. Our setback in using [node.js](#) as opposed to flask also taught us a valuable lesson in the importance of early design choices and sticking to them. Overall, we believe that we have delivered on the vast majority of sprint 1 requirements, but it is due to the previously mentioned reasons why there are some limitations to the planning of our first prototype. We aim to take these problems into consideration when looking forward to sprint 2, and calibrating our scope to a more realistic level.

Algorithmic Correctness

Evidenced by the passing of our tests, the correctness of our routefinding algorithm has been considered a success, and we are satisfied with this element of our project. We are in firm belief that we fully delivered this aspect of the application, as the routefinding algorithm does exactly what it was intended to do with minimal errors.

For the first sprint, we are pleased with the progress we have made with this measure of success. There are a few ways in which the algorithm behaves that could be improved upon in sprint 2, such as the way in which it deals with roundabouts. Currently, two very similar routes between points could be displayed that only differ when it comes to the way in which they go around a certain roundabout (clockwise or counter-clockwise). Small issues like these have been identified as low priority given the fact that we still have many more requirements to meet in sprint 2, but we will keep this in mind nonetheless. The actual correctness of the algorithm appears to be flawless.

Fairness and Harm Analysis

The main discussion surrounding this measure of success will be outlined in our ethical and legal considerations section. At risk of repeating ourselves, we will leave out the bulk of that discussion here. However, in summary, there is an ethical debate to be had around the morality of getting users to trust route scores. This application does not collect its own data and in no way does it promote one hundred percent authenticity when it comes to displaying information for the lighting, safety, elevation or any other description of a route between nodes. It is important to ensure the user knows this and they do not rely on the app in a way that might be deemed unsafe. To mitigate these potential issues, we have placed text disclaimers in our project to make sure the user is not confused and proceeds to travel in a safe manner. We believe that, thus far, this is a good step towards succeeding when it comes to preventing our users from harm.

Interpretability and Engagement

We have grouped both of these measures together as the issues surrounding them overlap heavily. Both the way in which the user is able to interpret the nature of the application and the way in which they return to use it repeatedly can be measured through stakeholder interaction. As of the end of sprint 1, communication with the sort of people who would benefit from our app (the “average walker”) has been scarce. This marks a large weakness in the early stages of our project; we have identified project requirements without going through an extensive stakeholder interview process, and consequently the entire architecture and implementation of our project has used these requirements as a guide. Through the process of sprint 2, we will aim to communicate more with potential users of our application and gain verbal feedback from them when it comes to the quality of the user experience and potential features they’d like to be included. The issue in doing this halfway through the project is that there could be many features users would want to add in order to make the app easier to use and more interpretable, promoting repeat usage. This will inevitably result in scope creep that we will have to control. Looking back on our first sprint, we wish we had communicated with stakeholders earlier on to get a better idea of what design would suit them. The rationale for not doing this was

because this entire sprint was oriented around functionality, whilst sprint 2 still stands as the opportunity to make the app user friendly. As it stands, our application is far from interpretable. The score breakdowns and inputs could be seen as cluttered and damaging to the UX. It does not encourage repeat engagement for these reasons. We definitely have not succeeded in these measures yet, but we were not planning to until later in the project. Hence, we are not behind schedule on delivering in these ways.

Potential Risks In Sprint 2

When discussing to what extent we have satisfied our measures of success, a few risks were outlined for the second sprint. These dangers are generally linked to the possibility of scope creep as we aim to make our application more user-friendly. We have a couple of requirements from sprint 1 that must now be completed in sprint 2, meaning that we will be developing more than initially planned in a shorter amount of time. In addition to the features we currently aim to add, stakeholder feedback could further complicate this matter by forcing us to go back and change previously implemented features for the benefit of the user. We could also be given extra features stakeholders wish to see from the app, which could add even more to our scope. In order to control all of these issues, it is imperative that we review the scope of our project in our weekly scrum meetings and be realistic about what we can achieve in the time remaining. After working on the user login system in sprint 1 when we didn't need to do it until a later point, we have learned first hand the dangers of not carrying out our priorities. The features that have been identified as project requirements must be implemented as such priorities, and stakeholder preferences must come second. This way, by the end of sprint 2, we will at the very least have everything we sought to include. There were a few instances in sprint 1 where we attempted to implement our features using tools that ended up not being optimal, losing us time by needing to change our implementation approach. We will combat this by deciding what tools we will use as part of the initial sprint 2 architecture, and we will firmly remain with these choices for the remainder of the project to maximise efficiency and reduce the danger of falling behind schedule. At the start of our project, we planned to change team roles between sprints. Now we have reached that point, it is important that we are careful with what new team roles are assigned, if any. This is because all team members have become very knowledgeable and immersed in their individual roles, so changing them drastically could create a new unwarranted learning curve at the start of the second sprint.

Sprint 2 Plans

In order to operationalise our plans for sprint 2, we have created an acceptance criteria for all remaining requirements. This can be viewed in the table below...

Sprint 2 Acceptance Criteria

Requirement Number	Requirement Description	Measurement Method
6	Side-by-side comparison of two routes on indicator breakdown and score.	Visual inspection of side-by-side comparison. Manual tests ensuring indicator breakdowns and scores work as intended.
7	At least 6 analytics views across the system.	Manual test ensuring that six individual analytical statistics can be viewed on the system.
9	Route segments should show indicator tooltips upon hovering.	Manual test ensuring that indicator tooltips are visible.
10	Admins must be able to create/edit missions.	Manual test creating and editing a mission and observing whether or not it appears for users.
12	Change log required for edits (who changed what and when).	Screenshots of a valid change log.
13	Role based access control.	Extensive manual tests as a user and as an admin, carrying out all available actions and ensuring they are all allowed as part of that role.
14	Mobile-friendly layout.	Screenshots/video capture of a working mobile user interface.
15	Users can complete preplanned "missions" between points assigned to them.	Manual test attempting to complete a user mission.

The mobile-friendly layout and admin mission creation/editing features are now part of sprint 2. Everything else is taken from the table of sprint 2 requirements in the problem identification stage. The completion of the requirements above is our final objective, and will mark the end of our project.

Team Responsibilities

In the early stages of our project, we assigned specific roles to each member of our team. A list of our responsibilities for sprint 1 can be found here...

William Jones - Report Writing / Documentation

Ned Davis - Data sourcing and route displaying
Ryoma Starling - Designed the Database and Database manager
Edward Lucaciu - Map design and integration with the database
Tom Mortimer - Testing, System Architecture, Deliverables
Benjamin Jones - Route-finding algorithm design and implementation
Sam Carpenter - HTTP Response Model and Server Implementation
Joseph Tarrant - User Interface, Front-end design and Implementation

Acknowledgements

Map tiles and data used in this analysis are © OpenStreetMap contributors, available under the Open Database License. The Map data has been displayed by Leaflet.js, an open-source Javascript API available under the BSD-2 Clause License.